

ParkVision 智能停车系统结题文档

文档说明：本文档用于对 ParkVision 智能停车系统的项目背景、需求分析、总体设计、核心实现、测试验证、项目成果与后续方向进行系统化总结。

适用对象：课程结题归档、项目复盘、后续二次开发与系统扩展。

文档基线：本文档内容基于当前仓库 `ParkVision-System` 在 `2026-06-14` 的实现状态整理而成。

摘要

ParkVision 是一个面向立体车库场景的智能停车系统项目，围绕“车辆入场、车位分配、AGV 调度、用户取车、费用结算、异常告警、设备联动与运营监控”构建完整的软件控制闭环。系统采用 Vue 3 + Vite 作为前端技术栈，使用 Spring Boot 作为后端核心框架，并以 H2 文件数据库作为默认持久化存储，实现了车主端 H5、管理后台、数字孪生大屏、AI 视觉中枢、履约调度中枢与系统状态页等多角色、多页面协同运行的综合应用形态。

与传统仅关注停车计费或静态台账管理的系统不同，ParkVision 更强调“业务状态可追踪、设备状态可观察、用户操作可闭环、AI 决策可解释”。在当前交付版本中，系统已经形成五个相互配合的核心业务组件：用户与权限组件、订单与状态机组件、调度与策略引擎组件、AI 与数智决策组件、数字孪生与监控组件。通过这些组件的协作，系统可以完成用户注册登录、车辆绑定、停车订单管理、预约锁位、VIP 优先取车、动态计费、AI 准入判定、入侵检测、数字孪生实时同步与后台审计追踪等关键功能。

从工程实现角度看，系统当前采用“模块化单体后端 + 前后端分离 + 持久化文件数据库 + SSE 实时推送”的落地方案。该方案兼顾了课程项目阶段的开发效率、部署复杂度和可用性要求，使系统能够在单机场景下稳定运行并为真实用户操作提供完整的软件支撑。同时，系统在结构设计上保留了向 MySQL、Redis、消息队列、真实边缘推理节点和设备网关演进的扩展空间，为后续工程化升级提供了明确基础。

在结题阶段，项目已经不再停留在本地开发与联调状态，而是进一步完成了服务器部署验证，使系统具备了持续运行和远程访问的基础条件。也就是说，ParkVision 当前不仅能够在开发环境中完成单机演示，还已经具备通过服务器环境承载前后端服务、面向外部浏览器进行访问的能力，这一点对项目从“课程作业”走向“可用原型”具有重要意义。

关键词

智能停车；立体车库；AGV 调度；数字孪生；AI 视觉识别；状态机；Spring Boot；Vue 3；SSE 实时推送；RBAC 权限控制

1. 项目概述

1.1 项目背景

随着城市核心区机动车保有量持续增长，传统平面停车场在土地资源利用率、车辆周转效率和高峰值吞吐能力方面逐渐暴露出明显瓶颈。为提升单位面积停车容量，立体车库、AGV 托举式车库等自动化设施被越来越多地引入实际场景。然而，硬件设施的自动化并不天然意味着整体系统的智能化。如果控制系统仍停留在“静态台账 + 简单排队 + 被动取车”的水平，那么车辆调度效率、用户体验和安全治理能力仍然难以满足复杂场景需求。

在实际业务中，立体车库系统通常面临以下几类问题：

1. 高峰期大量取车请求会导致排队时间显著增加，若缺乏预判与优先级控制，系统吞吐容易下降。

- 2. 车辆、车位、调度任务、设备状态与支付结算之间存在紧密耦合，任何一个环节失步都可能造成业务紊乱。
- 3. 车主端、运营管理端、调度监控端和安全值守端的需求差异明显，系统需要在统一平台下完成多角色协作。
- 4. 视觉识别、异常告警与应急控制不能只作为附属功能存在，而应与业务主链路形成闭环。
- 5. 如果系统缺乏实时推送、审计日志和统一状态管理，即便页面功能较多，也难以真正支撑连续运营。

ParkVision 正是在上述背景下设计与实现的。项目不把自己定位为单一的“停车收费页面”或“展示型数字孪生页面”，而是尝试构建一个能够承载真实停车业务流、设备状态流与安全决策流的软件控制中枢。

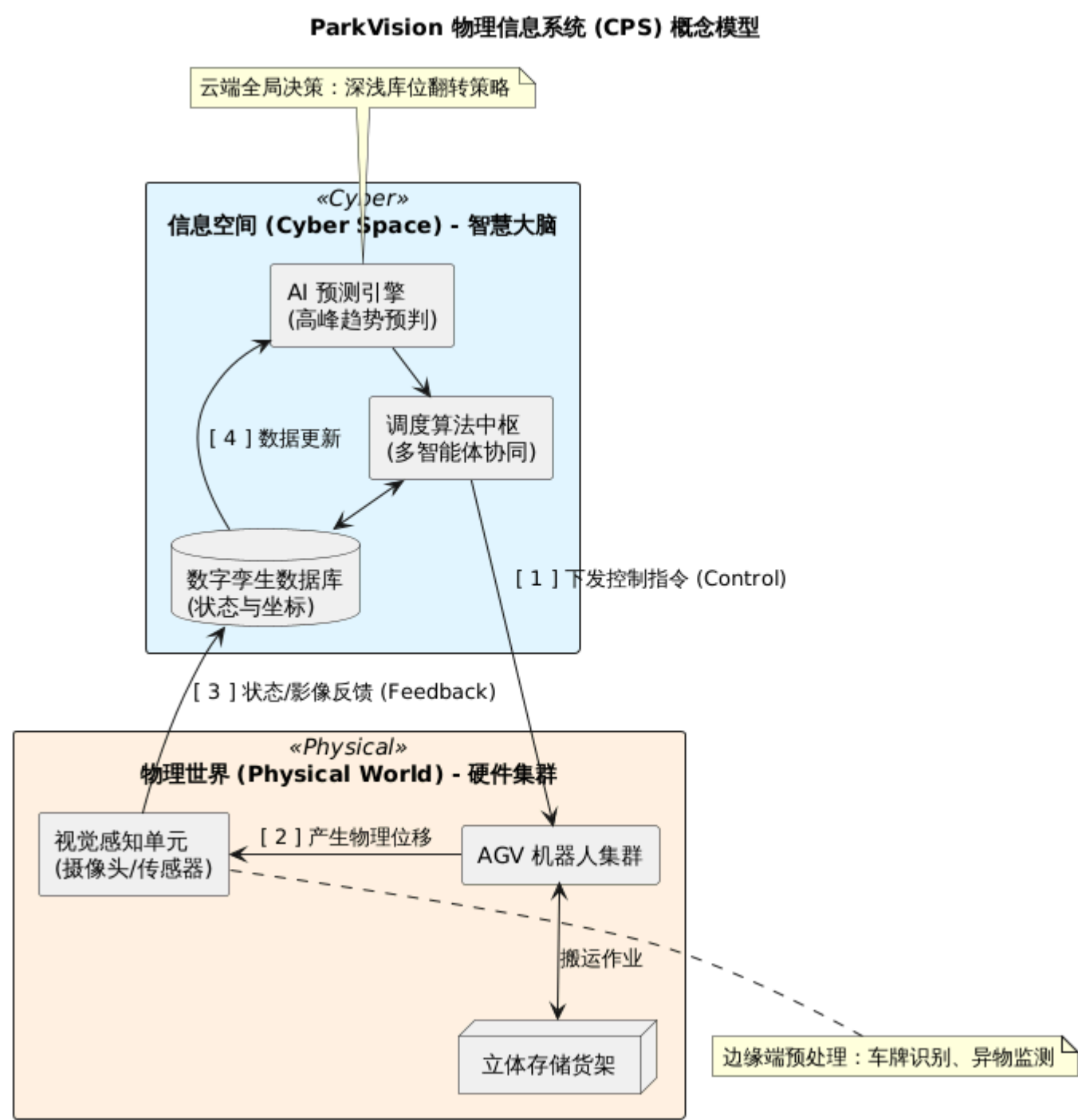


图 1-1 物理信息系统概念模型

1.2 项目目标

本项目的总体目标是构建一个面向立体车库场景的软件系统，使其同时具备用户服务能力、后台治理能力、实时监控能力与智能决策能力。具体目标包括：

目标方向	目标内容
面向车主	支持注册登录、车辆绑定、存车、取车、临时取物、预约锁位、账单查询、钱包充值、导航与智能问答
面向运营	支持订单管理、用户管理、计费规则配置、名单管理、报表生成、设备查看、审计追踪
面向调度	支持任务队列查看、AGV 遥测展示、预调度、VIP 优先取车、实时队列推进
面向监控	支持数字孪生大屏、系统节点健康状态、设备状态总览、急停控制与异常告警展示
面向智能化	支持视觉识别、准入判定、入侵检测、AI 对话代理、车流预测与运营摘要生成

1.3 项目定位与边界

需要明确的是，ParkVision 的当前交付版本是一个完整的智能停车软件系统，但它的边界主要位于“软件控制与业务管理层”，而不是硬件制造层或城市级交通基础设施层。当前系统已经实现了真实的业务数据流、状态流和权限控制，但以下内容不属于本次项目直接交付范围：

- 1. AGV 机械本体、PLC、电控柜、摄像头等硬件设备的制造与采购。
- 2. 单场站以外的多场站协同调度、跨园区统一运营。
- 3. 完整的生产级消息队列集群、Redis 分布式锁集群、高可用数据库主从架构。
- 4. 自研大模型训练与工业级视觉模型训练平台。
- 5. 真实支付渠道的收单清算链路。

因此，从真实可用性的角度看，ParkVision 当前更适合作为单场站软件中枢、课程项目原型与后续工程化扩展基础，而不是直接宣称已经完成大型商业化落地的最终版本。

不过需要特别说明的是，“原型”并不意味着“只能展示”。当前版本已经实现了真正的业务落库、权限限制、设备状态模拟、预约履约、账单生成和实时快照同步，系统中的大量功能并非一次性演示流程，而是可以被反复操作、持续运行并保留历史结果的。正因为如此，本项目在文档编写时需要同时强调两点：一方面，它仍有清晰的后续工程化提升空间；另一方面，它已经能够以较完整的软件形态对外提供服务，而不是停留在静态方案层面。

1.4 最终交付成果概述

基于当前仓库实现状态，本项目已经形成如下可交付成果：

成果类别	主要内容
前端应用	登录页、运营首页、数字孪生页、AI 视觉中枢、履约中枢、管理台账、系统状态页、车主端 H5
后端服务	16 个控制器、19 个服务类、统一仓储接口、JWT 认证、审计拦截、SSE 推送、定时任务机制
数据持久化	H2 文件数据库默认落地，提供 H2 与 MySQL 双 schema，支持本地持久化运行

成果类别	主要内容
核心业务能力	用户注册登录、车辆绑定、订单全生命周期、预约锁位、动态计费、AGV 调度、AI 视觉准入、告警与急停
工程支撑	一键启动脚本、Docker Compose 草案、本地 OCR 服务接入、后端与前端测试用例
文档沉淀	需求分析文档、系统设计文档、项目管理文档、系统实现文档、PPT 方案与本结题说明文档

1.5 服务器部署现状

除本地开发运行外，ParkVision 在结题阶段已经完成服务器部署验证。部署采用前后端分离思路：前端通过构建后的静态资源发布到服务器 Web 服务中，后端通过打包后的 Spring Boot 可执行程序运行在服务器进程中，并通过配置项注入数据源、JWT 密钥、AI 接口与 OCR 服务等关键参数。这样做的好处在于，前端页面能够以稳定的静态资源形式对外提供访问，而后端则可以独立维护业务逻辑、状态持久化与接口安全策略。

从仓库中现有的生产脚本可以看出，项目已经具备比较清晰的部署路径。`backend/scripts/package.ps1` 用于构建后端可执行 jar 与前端 `dist` 产物，`backend/scripts/start-prod.ps1` 用于以生产化方式启动后端服务，并支持通过环境变量设置数据库账号、数据库地址和 JWT 密钥。这说明项目在结题时已经考虑到了“如何部署”和“如何复现”两个现实问题，而不是只关注本地 IDE 中的运行效果。

服务器部署完成后，系统的使用方式也发生了变化：原本依赖开发机直接运行的前后端服务，已经可以通过部署环境被持续托管，用户只需要通过浏览器访问相应页面即可完成登录、业务操作和状态查看。对于一个课程综合项目而言，这意味着系统已经跨过了“仅开发者自己能跑”的门槛，具备了更接近真实应用的软件交付形态。

2. 需求分析与场景拆解

2.1 角色定义

ParkVision 不是单用户系统，而是一个天然多角色协同系统。不同角色看到的界面、可以触发的操作和关注的系统指标均不相同。

角色	身份定位	主要需求
车主	面向终端用户的 C 端角色	车辆绑定、查询订单、发起存车/取车、预约车位、查看账单、钱包充值、导航、问答咨询
管理员	场站运营管理角色	管理订单、账号、计费规则、名单、设备、报表与系统运行状态
调度人员	履约与队列管理角色	查看调度队列、介入 VIP 取车、执行预调度、观察 AGV 运行态势
安全值守人员	风险与应急处置角色	关注异常入侵、设备故障、告警列表和急停状态，必要时执行人工恢复
系统服务	后端内部逻辑角色	负责调度推进、状态同步、视觉决策、预测计算和快照广播

多角色协同带来的挑战不仅仅是页面数量增加，更在于系统必须清楚地划分“谁能看到什么、谁能操作什么、谁需要对什么结果负责”。例如，车主更关注订单、钱包和预约是否可用；调度人员关注队列推进、AGV 负载与执行效率；管理员则关心整体台账、规则配置、收入统计和风险告警。正因为这些角色在同一系统中承担着不同

职责，ParkVision 才必须采用统一认证、角色分流和服务端权限收敛的实现方式，否则很容易在后续迭代中产生权限混乱与数据越界问题。

2.2 关键业务场景

结合需求分析文档与当前实现代码，项目最终聚焦于以下关键业务场景：

1. 车主注册、登录与车辆绑定。
2. 管理员或车主发起车辆入场并自动创建停车订单。
3. 车主发起取车请求，系统将订单切换到调度状态并生成出库任务。
4. 车主进行临时取物，在不断单的前提下切换订单与车位状态。
5. 车主完成支付，系统执行结算并释放车位。
6. 车主提前预约车位，按预约到场后履约转为正式停车订单。
7. 系统根据调度策略执行高峰预调度、VIP 优先取车等逻辑。
8. AI 视觉识别入口车牌，对准入名单进行判定，必要时触发人工复核或急停联动。
9. 数字孪生页面与系统状态页实时消费后端快照，展示场站运行状态。
10. 管理员查看运营数据、设备信息、系统节点状态及审计记录。

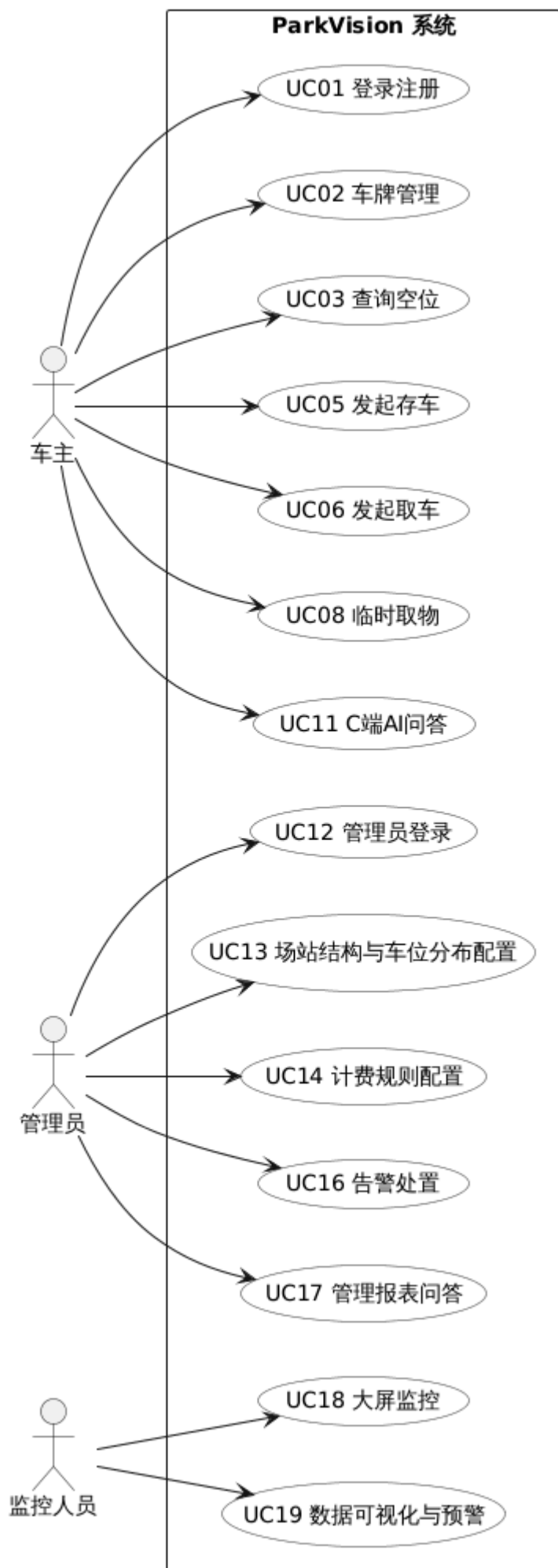


图 2-1 系统总体用例图

从这些场景可以看出，ParkVision 的需求并不是简单的“做几个页面、连几个接口”，而是围绕停车业务链路进行系统性拆解。每个看似独立的功能点，例如预约锁位、临时取物、AI 识别或 VIP 取车，最终都要回到统一的订单状态、车位状态与调度状态上来。因此，在需求阶段尽早把场景和参与者梳理清楚，是整个系统后续可以稳定实现的重要前提。

2.3 功能需求归类

从用户可见功能与后端实现两个角度看，系统需求可以归纳为五大组件域：

组件域	核心功能
用户与权限组件	登录、注册、角色分流、车主资料、车辆绑定、后台账号治理、权限限制、操作审计
订单与状态机组件	入场建单、重复入场校验、标准取车、临时取物、支付关闭、预约履约、异常处理
调度与策略引擎组件	队列展示、AGV 状态查看、预调度、VIP 优先、调度任务自动推进、动态计费预览
AI 与数智决策组件	AI 对话、视觉识别、门禁准入、黑白名单判定、入侵检测、车流预测、运营摘要
数字孪生与监控组件	KPI 汇总、预测曲线、设备总览、系统节点、数字孪生快照、实时流推送、急停控制

2.4 非功能需求

为了让系统具备真实使用价值，项目不仅关注功能是否存在，还重点考虑了性能、可靠性、安全性与可维护性等非功能属性。

非功能要求	具体体现
安全性	JWT 令牌认证、角色隔离、接口鉴权、写操作审计日志、名单判定与急停控制
一致性	订单、车位、调度、计费、设备与告警状态由服务层统一维护
实时性	使用 SSE 向前端推送数字孪生快照，调度与设备状态通过定时任务持续刷新
可观测性	设备事件、告警事件、系统节点状态、审计日志和运营指标可被后台查询
可扩展性	仓储接口抽象、H2/MySQL 双 schema、AI 服务与 OCR 服务配置化切换
可部署性	提供 <code>start.ps1</code> 、 <code>stop.ps1</code> 和 <code>docker-compose.yml</code> ，支持单机快速启动

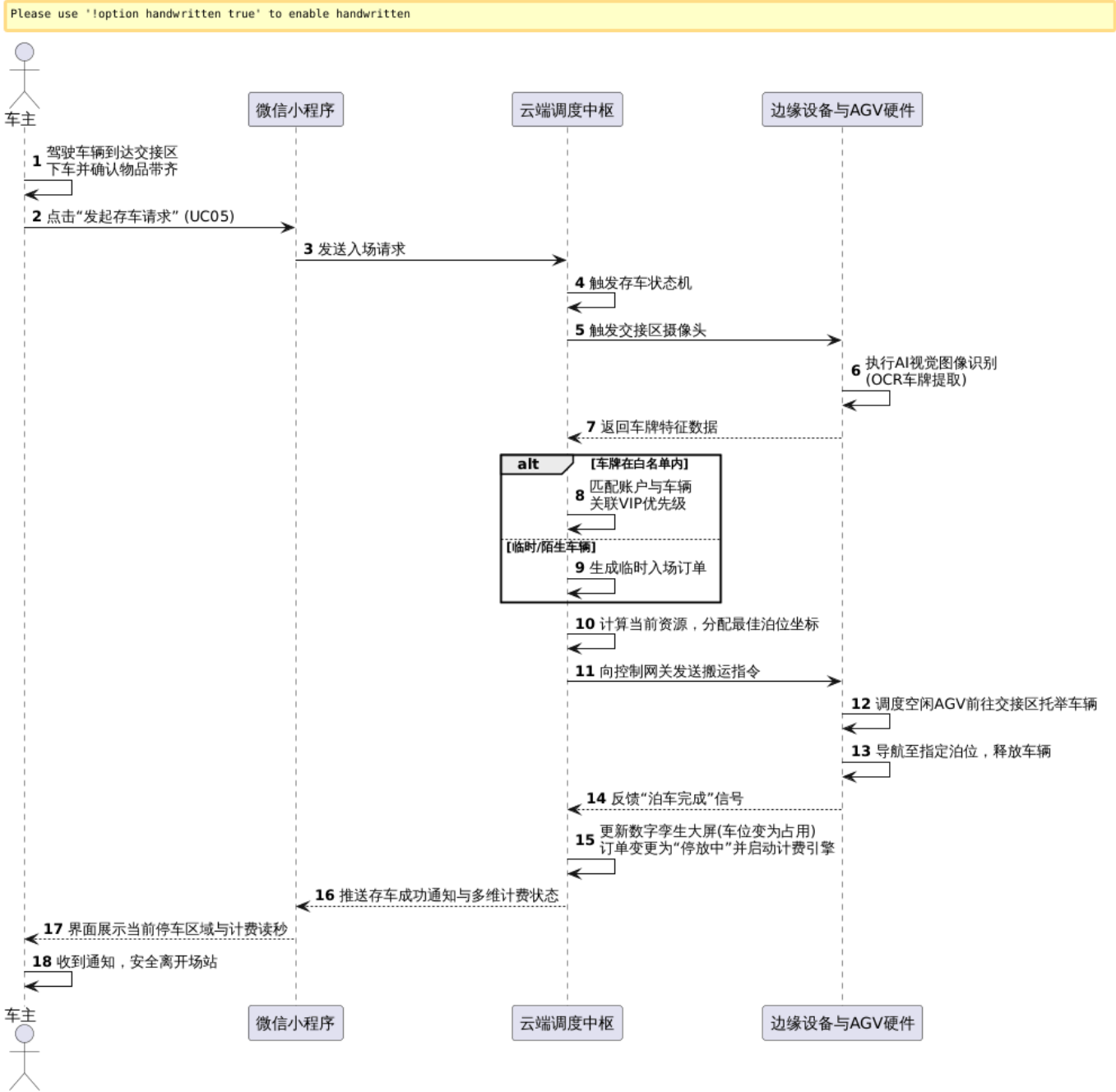


图 2-2 核心业务泳道图

非功能需求在本项目中的重要性不亚于功能需求本身。原因在于，智能停车系统不是一个“填表提交即可结束”的管理系统，而是与物理动作、用户等待时间、调度节奏和安全风险直接相关的控制系统。如果系统在一致性、实时性或安全性上存在明显缺口，那么即便页面功能齐全，也难以真正支撑真实使用。因此，本项目从需求分析阶段起就把非功能指标纳入核心设计约束，而不是放在最后作为附加项处理。

3. 系统总体方案与架构设计

3.1 总体架构说明

当前版本的 ParkVision 采用前后端分离架构：前端负责交互展示与多页面协作，后端负责统一的业务处理、数据持久化、权限控制与实时状态推送，AI 识别与大模型对话能力通过后端接口进行封装与桥接。

ParkVision 系统四层逻辑架构图

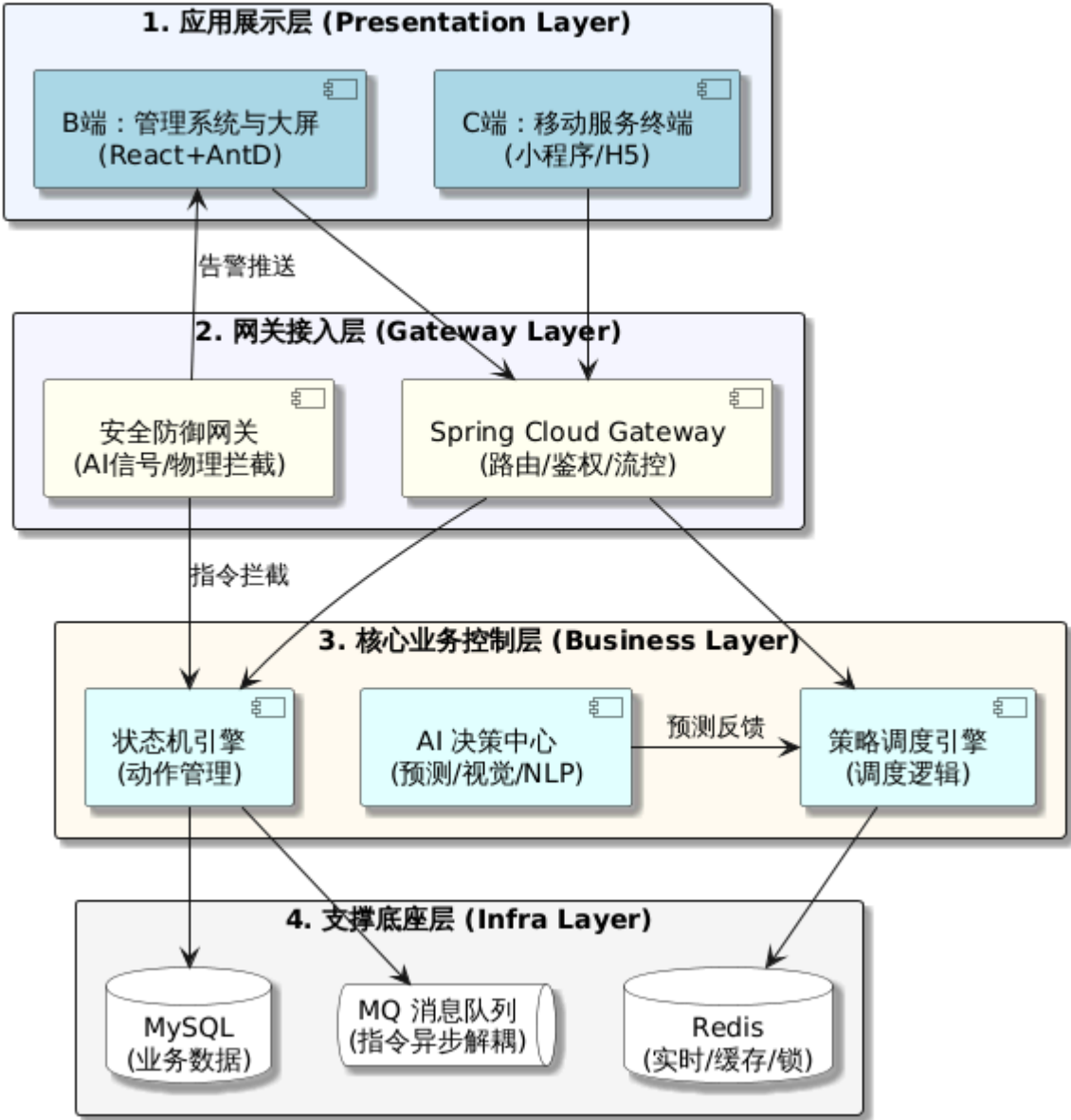


图 3-1 系统总体架构图

从实现结果来看，项目虽然在设计思想上借鉴了组件化、服务化和云边协同的模式，但当前交付版本并没有采用真正的分布式微服务集群，而是以“模块化单体后端”的方式实现。这样做的原因是：在课程项目周期内，统一进程部署更利于保证状态一致性、降低运维成本和提升整体可控性；同时，通过明确的 `controller/service/repository/domain` 分层和组件职责划分，系统仍然保留了后续拆分微服务的可能性。

进一步来看，当前架构方案的价值不在于“组件名是否足够前沿”，而在于各组件之间的边界是否清晰、数据流是否可追踪、关键状态是否可落盘。ParkVision 在总体方案上将前端交互、后端业务、AI 感知和监控推送分开组织，使系统既能承载用户操作，也能支撑后台治理与实时展示。这种结构使后续新增业务时无需推翻既有体系，只需要在既有组件边界内扩展功能即可。

3.2 前端架构设计

前端使用 Vue 3 + Vite 实现，采用单页应用方式组织系统页面，并使用 Vue Router 对不同角色进行导航控制。当前一级路由如下：

路由	页面	角色定位
/login	登录页	公共入口
/	运营首页	管理端
/twin	数字孪生页	管理端 / 监控端
/ai	AI 视觉中枢	管理端 / 安全侧
/dispatch	履约中枢	调度侧
/admin	管理台账	管理端
/system	系统状态页	管理端 / 运维侧
/owner	车主端 H5	车主

前端状态组织以 `frontend/src/stores/parkingStore.js` 为核心。该文件承担以下职责：

- 1. 统一维护前端全局状态，如登录状态、车主数据、订单数据、设备数据、数字孪生数据和管理员台账数据。
- 2. 将各页面所需的数据加载逻辑收敛到统一状态仓储中，避免页面间重复发起无序请求。
- 3. 提供登录、注册、加载车主数据、加载后台数据等状态驱动函数。
- 4. 为图表页、孪生页、后台页和车主端页提供同一份数据基线，保证不同视角下信息口径一致。

从系统结构角度看，前端并不是若干独立页面的随意拼接，而是以统一状态源组织多视图协同，这是后续构建大型管理系统时非常重要的工程实践。

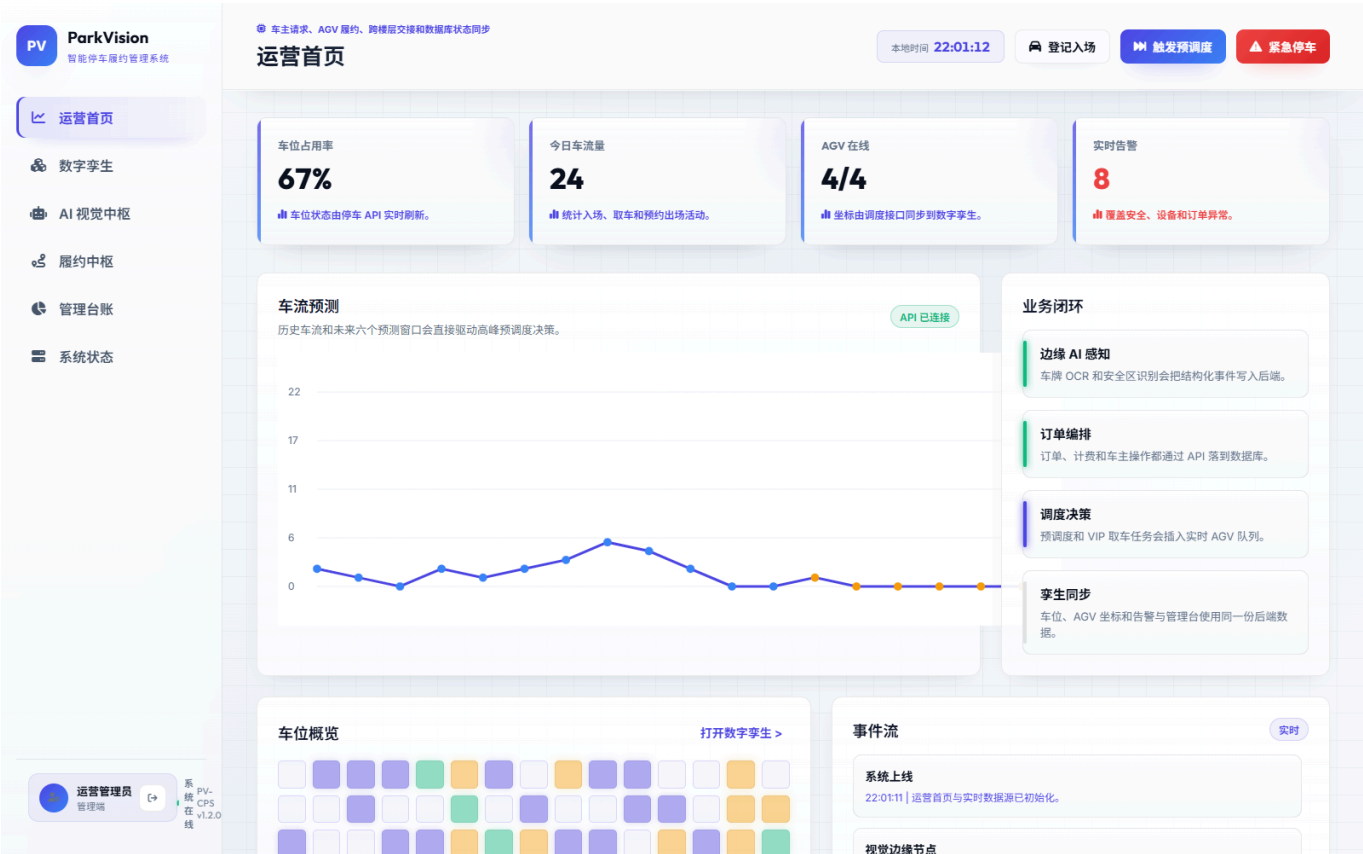


图 3-2 运营首页



图 3-3 车主端页面

前端页面设计上，项目并没有把管理端和车主端分别做成完全脱离的数据孤岛，而是尽可能围绕统一的业务状态组织视图。这样一来，同一笔订单在车主端、管理台账、调度中枢和数字孪生中虽然表现形式不同，但底层依据的是同一份后端数据。这种“多页面、多角色、同源状态”的设计，能够显著降低页面间信息不一致的问题。

3.3 后端架构设计

后端采用 Spring Boot 3.4.1，实现了标准的分层结构：

```
com.parkvision.cps
  controller/    REST 接口层
  service/       业务逻辑层
  repository/    数据访问抽象层
  domain/        领域对象与状态枚举
  dto/           前后端数据传输对象
  common/        统一响应与异常处理
  config/        配置类
  security/      JWT 认证与安全过滤
  web/           SSE 推送辅助组件
```

其中：

1. **controller** 负责暴露 REST 接口，处理请求入参与响应结构。
2. **service** 负责核心业务逻辑，是状态变更与规则计算的主要发生地。
3. **repository** 提供统一的数据访问接口，隔离业务层与底层存储实现。
4. **security** 负责 JWT 解析、接口鉴权和安全策略配置。
5. **web** 目录中的 **TwinStreamHub** 与 **TwinBroadcastService** 共同构成了数字孪生实时推送能力。

后端分层的另一个现实价值，是能够把接口层与业务层的关注点彻底分开。控制器只负责接收请求、返回响应，服务层只关心规则与状态变更，仓储层只关心数据读写。这样的分工在项目规模较小时可能看起来有些“重”，但当系统引入权限控制、预约履约、计费拆分、实时快照和 AI 决策后，清晰分层会成为降低维护成本的关键手段。

3.4 仓储抽象与持久化设计

后端的 **ParkVisionRepository** 是系统持久化设计的关键抽象。它统一暴露了停车位、订单、预约、客户账户、车辆档案、支付记录、账单组件、名单、识别事件、系统节点、审计日志、AGV、设备和告警等核心实体的读写方法。这意味着业务服务层无需直接依赖数据库细节，而只依赖仓储接口定义。

当前默认持久化方式为 JDBC + H2 文件数据库。**application.yml** 中的默认数据源如下：

1. 数据库类型：H2 文件库。
2. 数据库存储路径：**backend/data/parkvision.***。
3. 初始化脚本：**schema-h2.sql**。
4. 兼容迁移能力：同时提供 **schema-mysql.sql**，为后续切换到 MySQL 提供基础。

这套设计既保证了本地单机环境下的数据可持久保留，也为未来扩展到更标准的生产数据库奠定了结构基础。

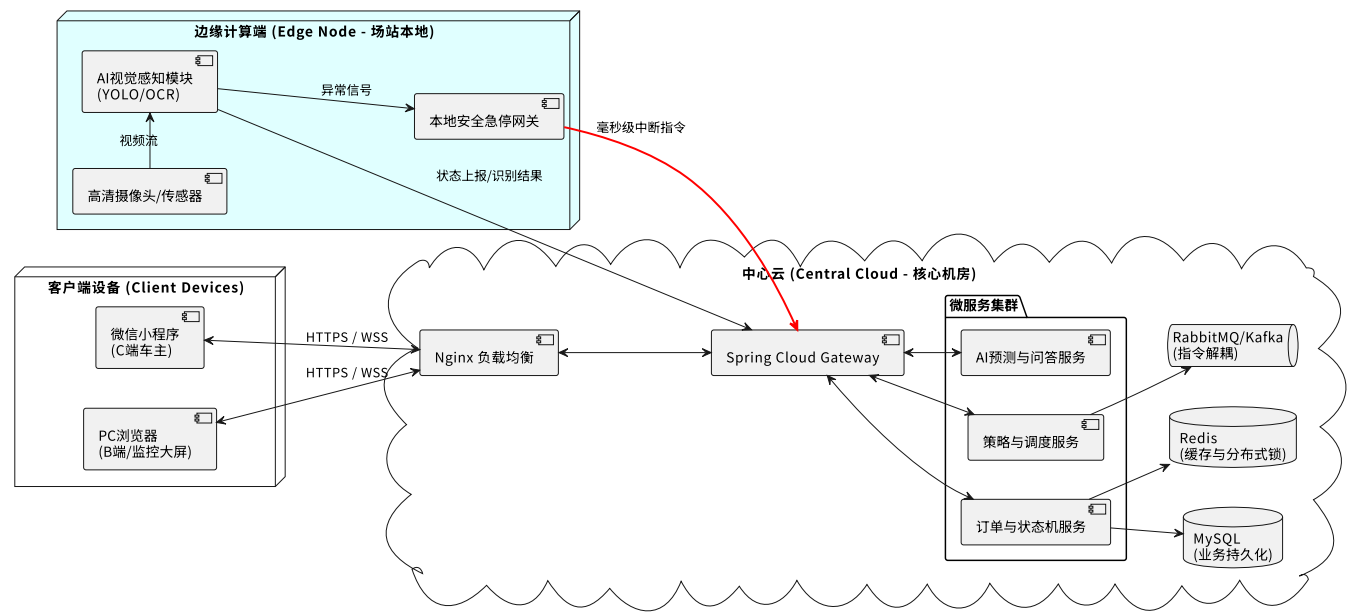


图 3-4 系统部署视图

在本地阶段使用 H2 文件库的一个现实好处，是大幅降低了项目复现门槛，使任意评审或开发成员在没有额外数据库环境的前提下，也能启动完整系统并保留历史业务数据。与此同时，仓库中又保留了 MySQL schema 和生产脚本，这说明项目并未把当前实现封死在本地演示场景内，而是预先考虑了后续迁移的工程路径。

3.5 实时通信机制

系统在实时数据更新方面采用了两类机制：

1. 普通业务接口使用 REST API 进行请求/响应通信。
2. 数字孪生与监控大屏使用 SSE 进行服务端单向推送。

后端通过 `TwinBroadcastService` 周期性构建统一快照，快照内容包括：

- 首页 KPI 汇总；
- 全部车位状态；
- 全部 AGV 遥测；
- 调度队列；
- 急停状态；
- 当前时间戳。

这些数据通过 `TwinStreamHub` 持有的 `SseEmitter` 集合广播给所有已连接前端页面。与纯轮询方式相比，这种方案可以在不引入 WebSocket 复杂握手与双向协议管理的前提下，较好地满足只读型大屏推送场景的需要。

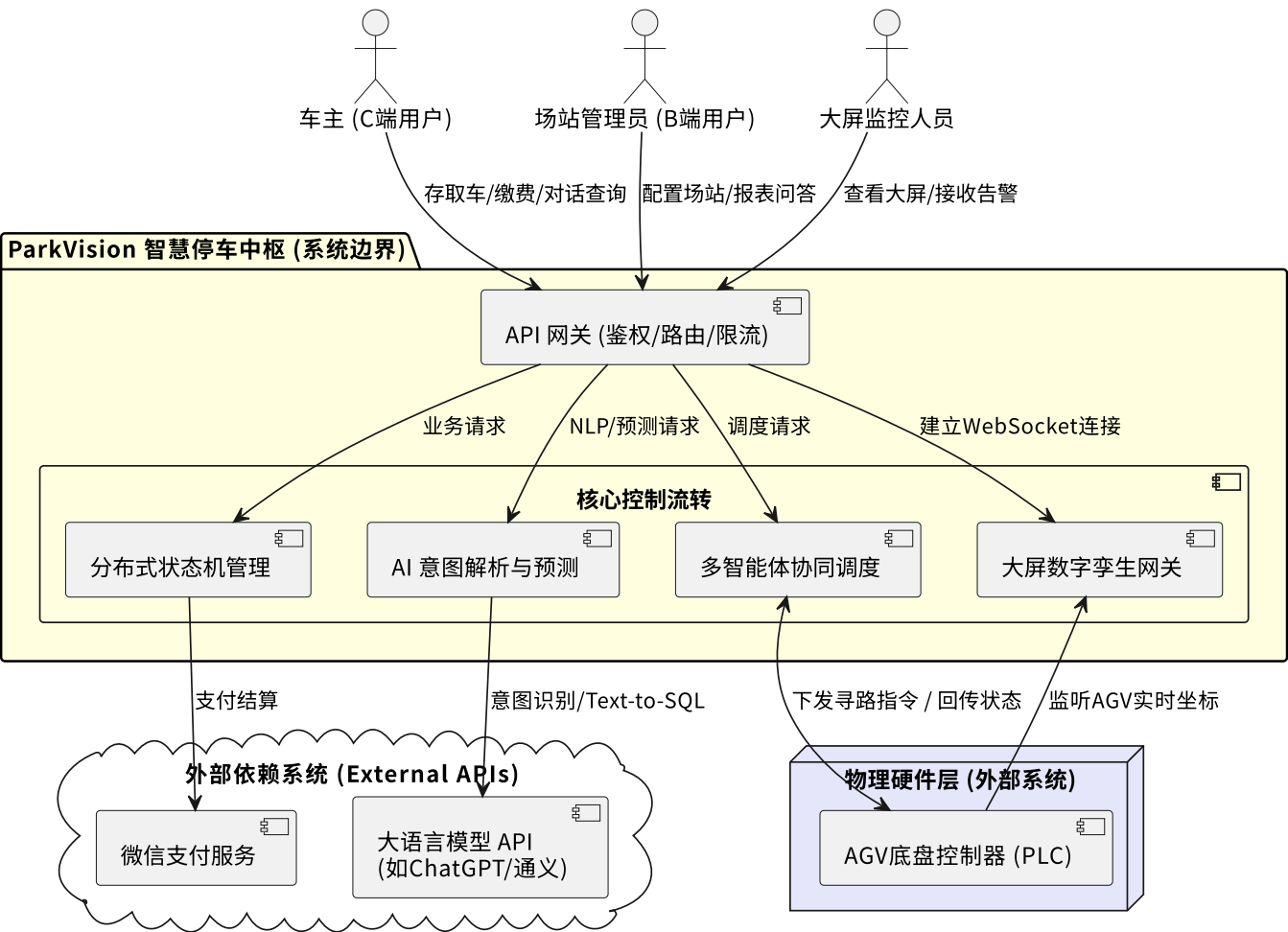


图 3-5 系统逻辑架构图

3.6 部署结构与运行方式

项目当前提供两种主要运行方式：

- 1. 本地脚本启动：使用 `start.ps1` 一键启动前端、后端及本地 OCR 服务。
- 2. 容器化草案部署：使用 `deploy/docker-compose.yml` 启动前端和后端容器。

在本地脚本方案中：

- 前端默认地址：`http://localhost:5173`
- 后端默认地址：`http://localhost:8080`
- H2 控制台：`http://localhost:8080/h2-console`
- 日志目录：`logs/`

此外，服务器部署完成后，系统运行不再依赖开发机保持前台窗口常驻。前端构建产物可以由服务器端静态资源服务持续托管，后端则以独立进程方式提供 REST API 与实时推送接口。这使得系统具备了更稳定的对外运行形态，也为后续多人协同试用、远程演示和持续调试提供了更方便的环境。

4. 数据模型与领域设计

4.1 核心实体说明

ParkVision 的领域模型围绕“用户—车辆—订单—车位—调度—支付—设备—告警”展开。当前主要实体如下：

实体	作用说明
CustomerAccount	车主账户档案，保存基本身份信息、会员等级和钱包余额
VehicleProfile	车辆档案，记录车牌、归属用户、能源类型、准入状态等
ParkingOrder	停车订单核心实体，承载入场、在库、取车、支付等生命周期
ParkingSlot	车位实体，记录位置与当前占用状态
Reservation	预约锁位记录，支持创建、取消、履约和过期释放
DispatchTask	调度任务实体，表示入场存车、预调度、标准取车、VIP 取车等动作
AgvUnit	AGV 实体，记录坐标、电量、模式、载车状态和当前任务
PricingRule	计费规则，定义免费时长、首小时费用、每小时费用、封顶价与高峰倍率
PaymentTransaction	支付记录，保存订单结算后的支付信息
OrderBillingComponent	账单组成项，记录基础停车费、高峰调节费、充电费、VIP 优先费等
AccessListItem	黑白名单 / 观察名单条目，用于门禁准入判定
RecognitionEvent	视觉识别事件，用于记录车牌识别与识别结论
AlertEvent	告警事件，记录门禁拒绝、异常行为、系统高风险信息等
DeviceEvent	设备事件，记录设备状态变化与动作结果
SystemNodeStatus	系统节点状态，用于描述核心节点健康度、负载与风险级别
AuditLog	操作审计日志，记录写接口调用的操作者、路径和结果

4.2 核心实体关系

从数据关系上看，系统存在以下几个重要关联：

- 1. 一个 CustomerAccount 可以拥有多辆 VehicleProfile。
- 2. 一个 VehicleProfile 在任意时刻最多只能对应一个未完成的 ParkingOrder。
- 3. 一个 ParkingOrder 必然绑定一个 ParkingSlot，并可能关联多个 OrderBillingComponent 和一条 PaymentTransaction。
- 4. 一个 Reservation 可以在履约时转化为一笔正式 ParkingOrder。
- 5. 一个 DispatchTask 与订单和车位紧密相关，并在执行时分配给某个 AgvUnit。
- 6. 一次视觉识别会生成一条 RecognitionEvent，并可进一步触发 AlertEvent、ParkingOrder 或 DeviceEvent。

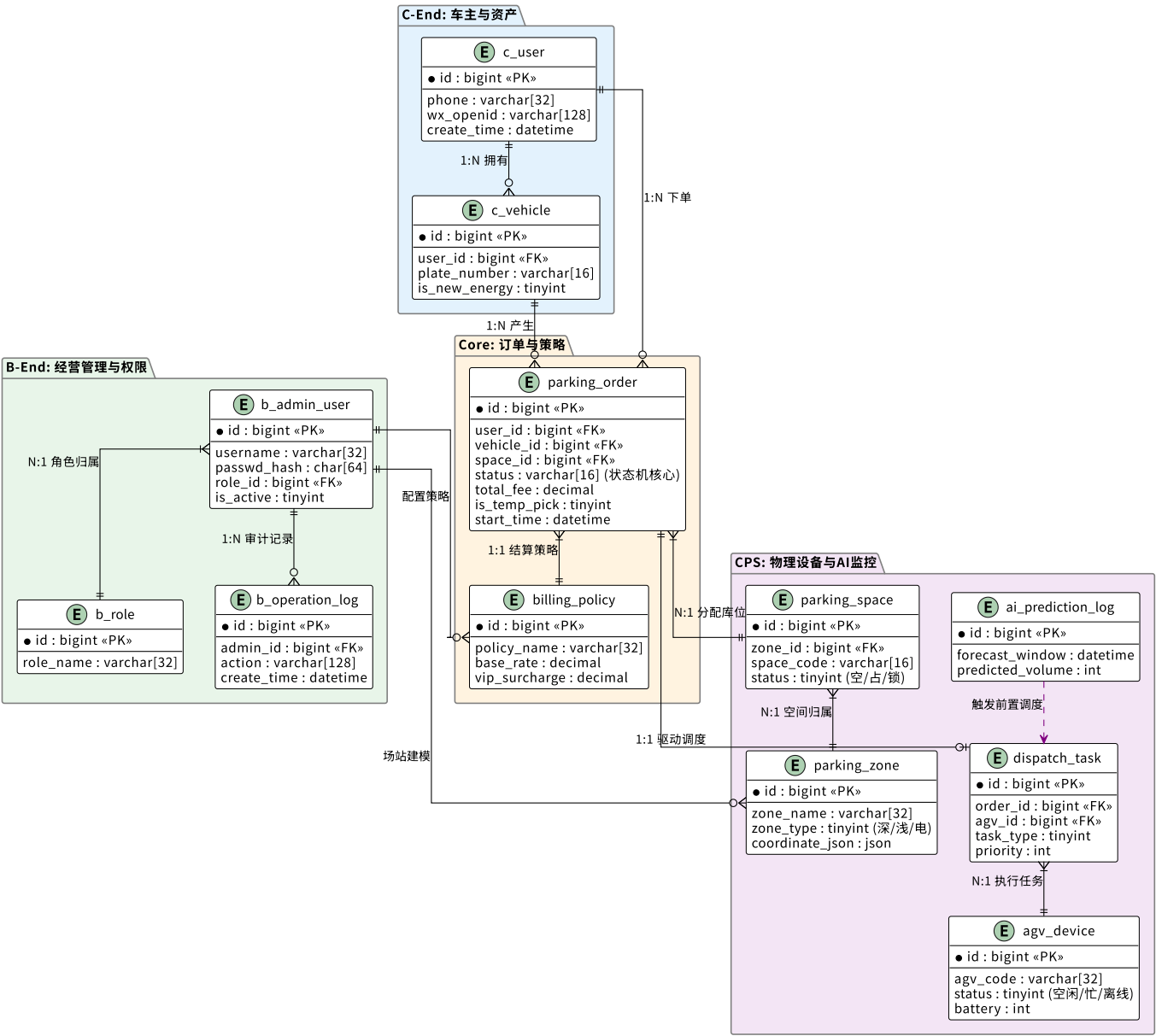


图 4-1 系统核心实体关系图

与很多只关注“把表建出来”的课程项目不同，ParkVision 在数据模型设计中更强调实体之间的业务约束关系。例如，一辆车不能同时拥有多个未完成订单，一个预约如果履约成功就必须转化为正式订单，一个支付记录必须能够追溯到对应订单与账单明细。这种关系设计使数据模型不只是静态存储结构，更成为业务规则的底层载体。

4.3 状态设计

系统中最重要的状态设计包括订单状态、车位状态和调度任务状态。

4.3.1 订单状态

订单状态围绕停车全生命周期组织，目前主要包括：

- **PARKED**：车辆已入库；
- **RETRIEVING**：用户发起取车，等待调度出库；
- **TOUCHING**：用户发起临时取物；
- **FINISHED**：订单已结算完成。

4.3.2 车位状态

车位状态主要包括：

- **EMPTY**：空闲；
- **OCCUPIED**：普通占用；
- **CHARGING**：充电占用；
- **BUFFER**：缓冲区状态，通常用于出库或临时取物阶段。

4.3.3 调度任务状态

调度任务的推进过程主要包括：

- **QUEUED**：已排队；
- **IN_PROGRESS**：执行中；
- **DONE**：已完成。

状态建模的意义在于：页面展示、后台统计和设备联动都不直接操作“页面变量”，而是围绕状态转换规则展开，从而提升整个系统的可控性与可追踪性。

这一点在实际开发中非常关键。因为一旦系统后续继续增加“预约保留中”“支付处理中”“故障隔离中”等中间状态，如果底层一开始没有建立清晰的状态模型，就会导致页面逻辑、接口逻辑和数据库状态各说各话。当前版本虽然状态集合还不算极度复杂，但已经为后续扩展保留了良好的结构基础。

5. 核心模块实现说明

5.1 用户与权限组件

用户与权限组件是系统入口能力的基础，其核心类包括 **AuthController**、**AuthService**、**OwnerController**、**OwnerService**、**AdminUserController**、**AccountAdminService**、**SecurityConfig**、**JwtService** 和 **AuditInterceptor**。

该组件完成了以下能力：

1. 管理员与车主两类身份的统一登录。
2. 车主自助注册，并在一次事务中完成账号、客户档案和首辆车信息的创建。
3. 基于 JWT 的无状态认证。
4. 路由级与接口级角色隔离。
5. 车主数据的归属校验。
6. 后台账户治理与状态控制。
7. 写操作审计日志自动记录。

在实现细节上，**AuthService.register()** 会校验用户名、显示名和车牌信息的有效性，在确认不存在重复账号和重复车辆后，同时写入 **CustomerAccount**、**VehicleProfile** 和登录用户信息，并返回 JWT 令牌。这意味着系统并不是先创建前端用户，再由管理员补录信息，而是从一开始就建立了“账号—人—车”的闭环关联。

安全配置方面，**SecurityConfig** 对 **/api/auth/**** 等公共接口开放访问，对 **/api/admin/**** 和 **/api/system/**** 进行管理员角色限制，并对其余 **/api/**** 统一要求认证访问。与此同时，**AuditInterceptor** 会在所有写请求完成后记录审计日志，从而建立起后台治理所需的责任追踪链路。

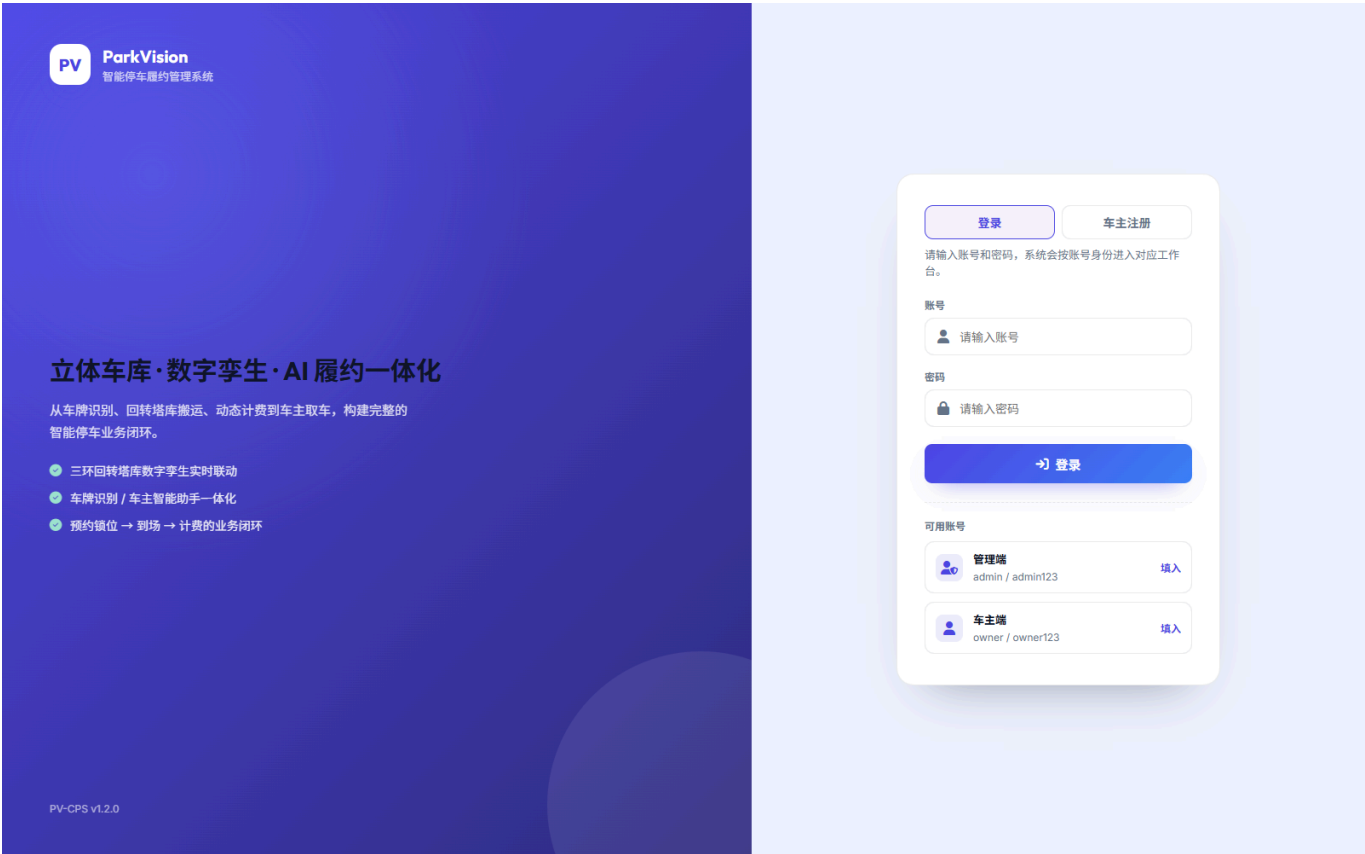


图 5-1 登录页

从真实系统角度看，这个组件的意义不只是“用户能不能登录”，而是它决定了系统的身份边界是否清晰。只有当车主、管理员、安全值守人员和调度人员的职责在服务端被严格区分，后续的订单查询、账号治理、设备控制和报表访问才能建立在可信前提上。

5.2 订单与状态机组件

订单与状态机组件是系统业务闭环的核心，其核心类为 `OrderController`、`OrderService`、`ReservationService` 和相关 DTO / Domain 类。

其主要功能包括：

- 1. 随机模拟入场与指定车牌入场。
- 2. 重复入场拦截。
- 3. 自动分配空闲车位。
- 4. 根据车辆属性设置 `occupied` 或 `charging` 车位状态。
- 5. 发起标准取车流程。
- 6. 发起临时取物流程。
- 7. 订单支付关闭与最终结算。
- 8. 预约锁位创建、取消、履约与自动过期释放。

具体来说，`OrderService.entryForPlate()` 在处理入场请求时，会先检查该车牌是否已存在未完成订单，防止同一辆车重复入场。通过校验后，系统自动查找可用车位，保存 `ParkingOrder`，并同步创建入场调度任务。之后，`changeStatus()` 会在用户取车、临时取物和支付完成等不同状态切换时同步更新车位状态和设备事件。

这种设计的关键价值在于：订单不是独立的数据表记录，而是整个停车业务链路的主线实体。车位占用、调度任务、计费结果和用户视图都围绕同一订单状态变化展开，因此状态机设计是否严谨，直接决定系统能否稳定运行。

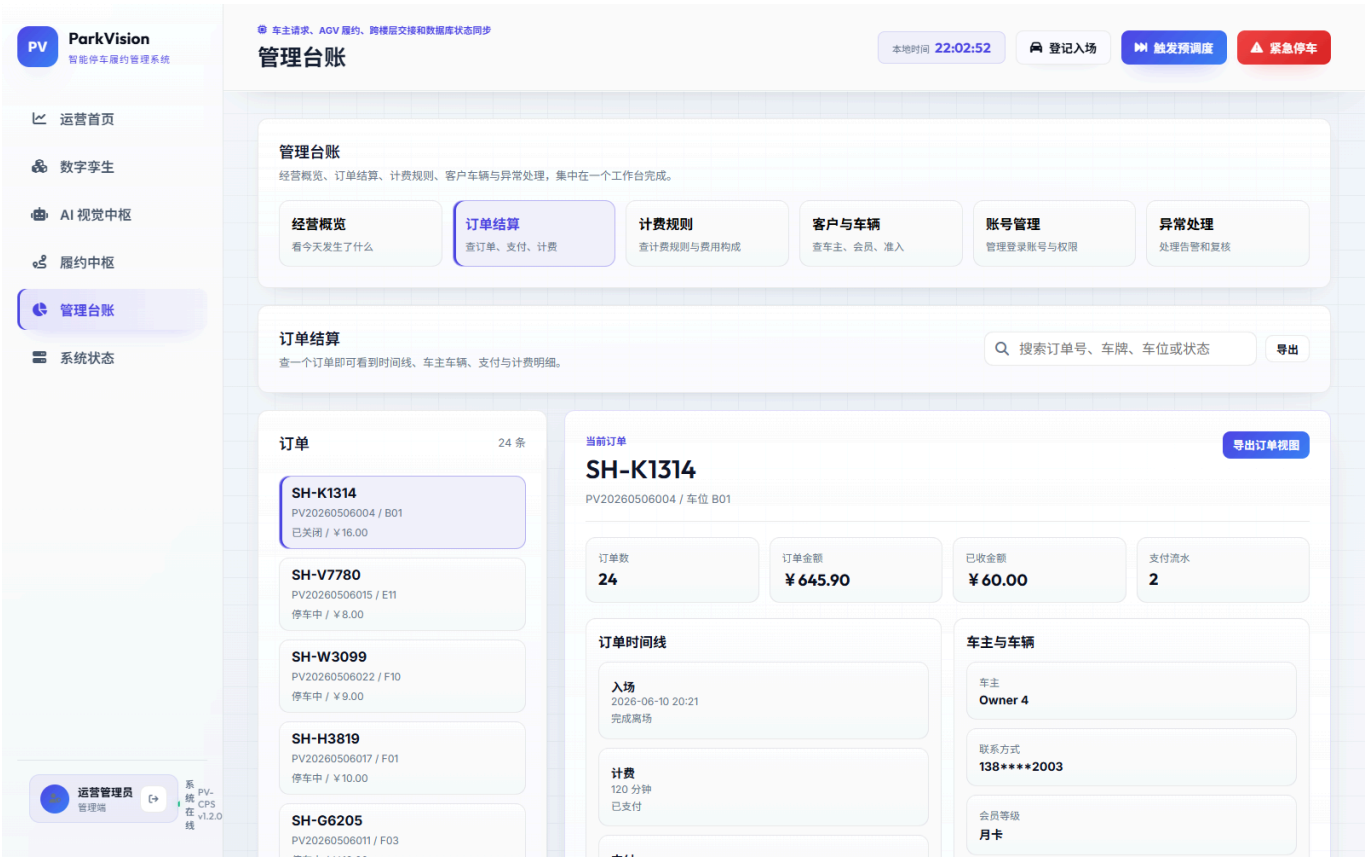


图 5-2 订单与状态机相关页面

进一步说，订单与状态机组件实际上承担了“业务骨架”的职责。用户是否能看到正确的当前订单、调度系统是否能接续执行、账单是否能正确生成、车位是否能在完成后被及时释放，这些都要依赖状态机是否稳定工作。因此，该组件虽然看上去是一个后端服务模块，但它对全系统的影响是贯穿式的。

5.3 调度与策略引擎组件

调度与策略引擎组件由 `DispatchController`、`DispatchService`、`PricingController`、`PricingService`、`BillingService` 等类共同完成。

该组件当前已经落地的能力包括：

- 1. 调度队列查询。
- 2. AGV 遥测状态展示。
- 3. 高峰预调度移库。
- 4. VIP 优先取车。
- 5. 调度任务自动推进。
- 6. AGV 空闲充电与待命逻辑。
- 7. 动态计费预览。
- 8. 规则驱动结算。

其中，`DispatchService.preDispatch()` 会优先选择深库位或占用车位执行预调度，将其切换到 `BUFFER` 状态并生成调度任务；`DispatchService.vip()` 则会在取车场景下调整订单状态与车位状态，并插入高优先级

任务。更重要的是，`DispatchService.advance()` 通过定时任务周期性推进调度队列进度，为正在执行的任务分配空闲 AGV、推进进度条、移动坐标、更新电量，并在任务完成时复位 AGV 状态。

计费部分采用规则驱动方式。`PricingService.quote()` 会根据当前车辆属性、停车时长、峰谷时段、充电情况和 VIP 优先费生成费用明细，`BillingService` 则负责最终结算与账单组件保存。这意味着系统在“用户看到的费用预览”和“最终实际结算的费用”之间建立了统一的计算口径，减少了业务分叉。

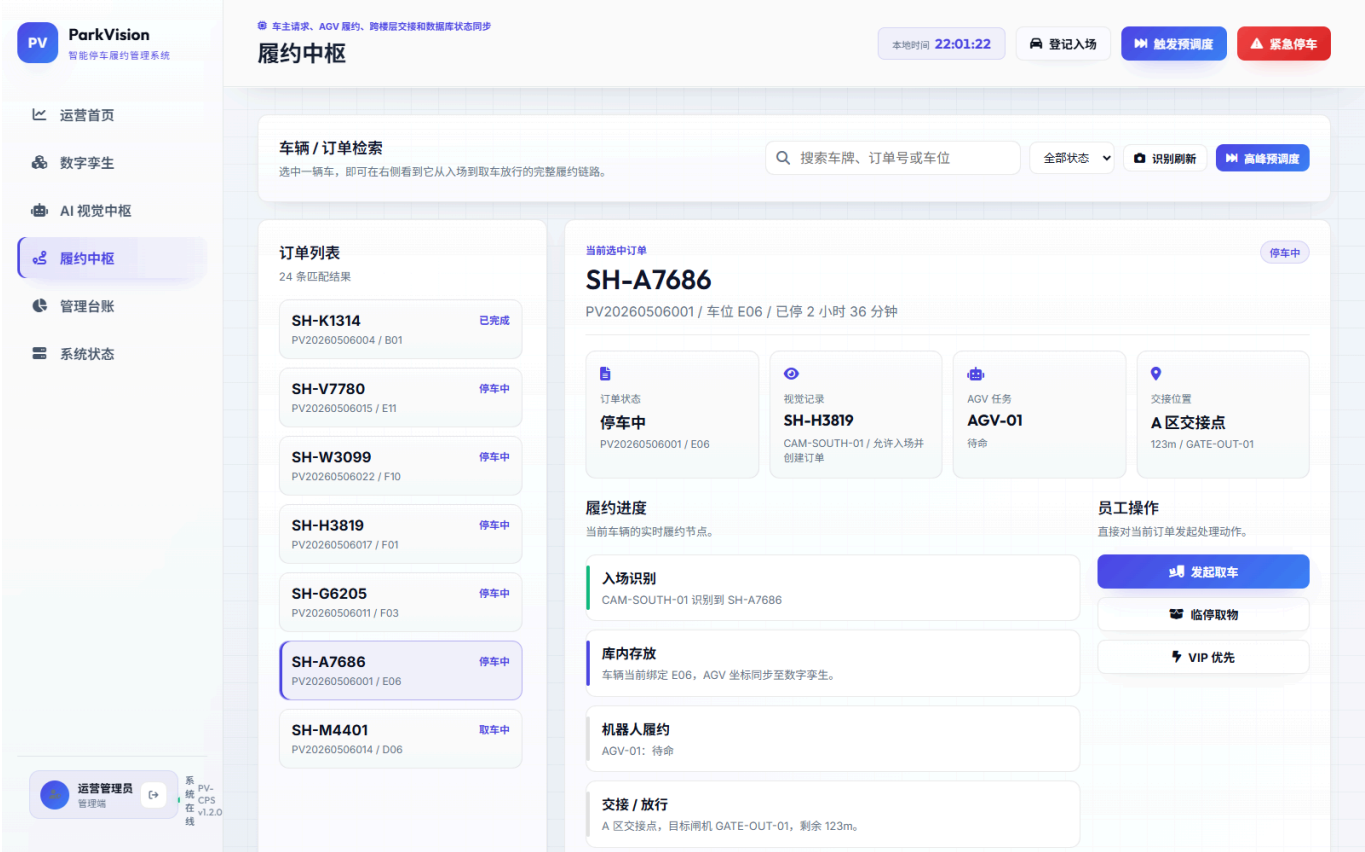


图 5-3 调度与策略引擎页面

这一组件的工程意义在于，它把抽象的业务意图翻译为具体的执行逻辑。对于用户来说，“我要取车”只是一次点击；但对系统来说，它意味着要重新判断订单状态、车位状态、队列优先级、AGV 可用性和当前设备负载。调度组件正是承接这些复杂关系的中枢。

5.4 AI 与数智决策组件

AI 与数智决策组件包括 `AiController`、`AiChatService`、`VisionController`、`VisionService`、`AccessControlService`、`PlateRecognitionService` 和 `DashboardService` 中与预测、报表相关的部分。

其核心能力如下：

- 1. AI 对话模型状态探测。
- 2. 基于 OpenAI 兼容接口的服务端 AI 对话代理。
- 3. 视觉识别预览。
- 4. 闸口识别联动入场。
- 5. 黑白名单与观察名单准入判定。
- 6. 入侵检测与急停联动。
- 7. 车流预测数据生成。
- 8. 运营摘要与报表文本生成。

VisionService 是该组件的关键实现。它在处理识别请求时，不会简单地将“识别到车牌”直接等价于“允许入场”，而是把视觉结果与 **AccessControlService** 的名单判定结合起来，输出 **ALLOW**、**REVIEW** 或 **DENY** 三类结论。如果识别结果是允许且请求为闸口入场模式，则系统直接调用 **OrderService.entryForPlate()** 创建正式入场订单；若识别到黑名单车辆、观察名单车辆或入侵行为，则系统会分别生成拒绝、人工复核或急停联动结果。

这种做法体现出一个关键原则：AI 负责提升系统感知与决策效率，但不应绕开业务规则体系。最终放行与否，必须建立在可解释的规则基础上。

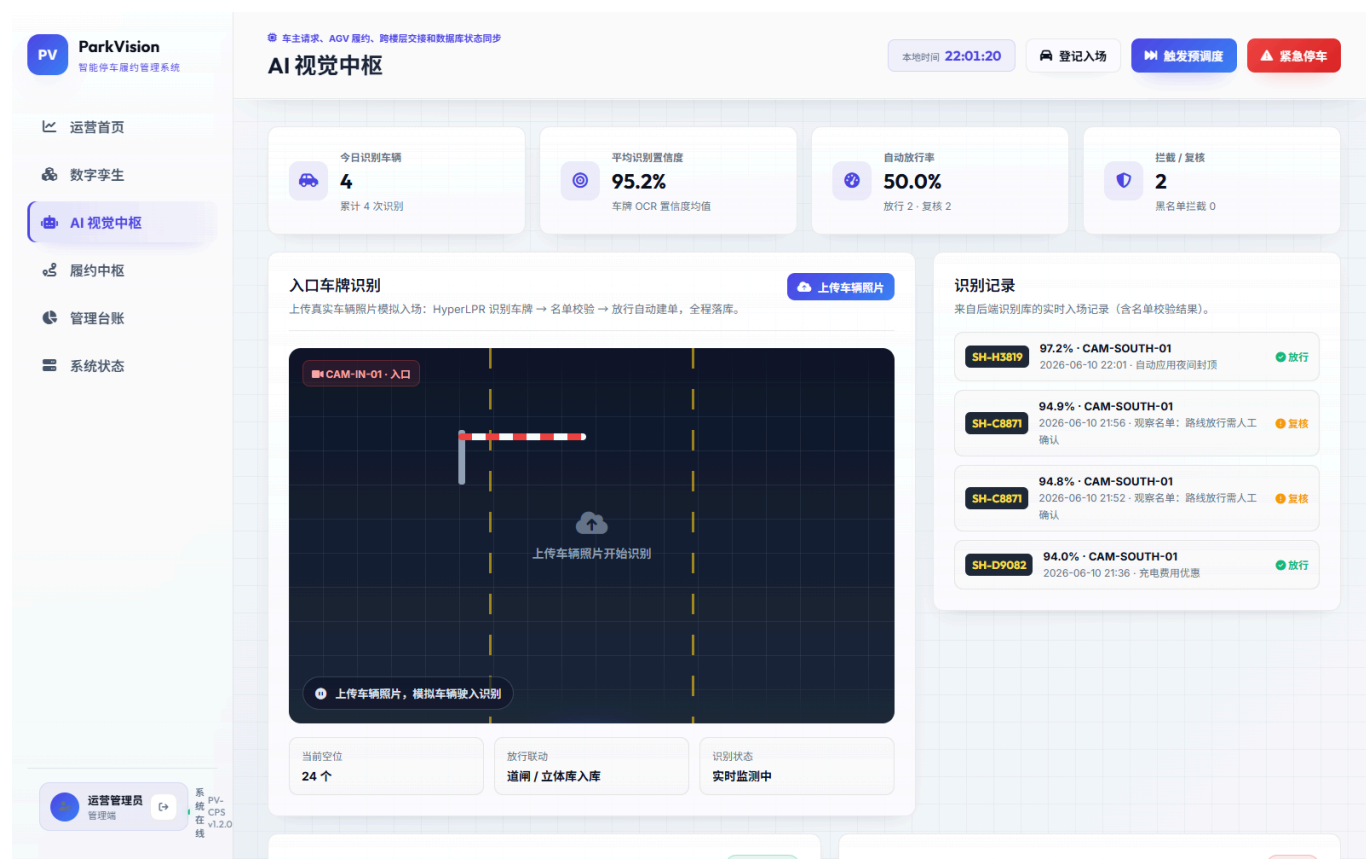


图 5-4 AI 与数智决策页面

这类设计使系统的智能化能力不是“额外附赠的高级功能”，而是嵌入在主业务流中的辅助决策能力。AI 模型负责理解、识别和预测，但最终动作仍要遵守系统的准入规则、状态约束和人工复核机制。这样既能发挥 AI 的效率优势，也能控制高风险场景下的误判成本。

5.5 数字孪生与监控组件

数字孪生与监控组件由 **DashboardController**、**DashboardService**、**TwinStreamController**、**TwinBroadcastService**、**TwinStreamHub**、**SystemController**、**DeviceController** 等构成。

该组件负责实现：

1. 首页 KPI 汇总。
2. 历史车流与未来预测曲线。
3. 数字孪生统一快照构建。
4. SSE 实时广播。
5. AGV、车位、调度队列的同步展示。
6. 系统节点健康状态查询。

7. 设备总览、设备事件和安全状态展示。
8. 急停与人工恢复控制。

`TwinBroadcastService.buildSnapshot()` 会把运营首页摘要、车位数据、AGV 数据、队列数据和急停状态合并成一个统一快照，再由定时广播任务推送给所有连接的前端页面。这样，数字孪生页面看到的不是本地虚拟动画，而是后端真实业务状态的投影。监控页面与数字孪生页面由此建立了数据一致性基础。

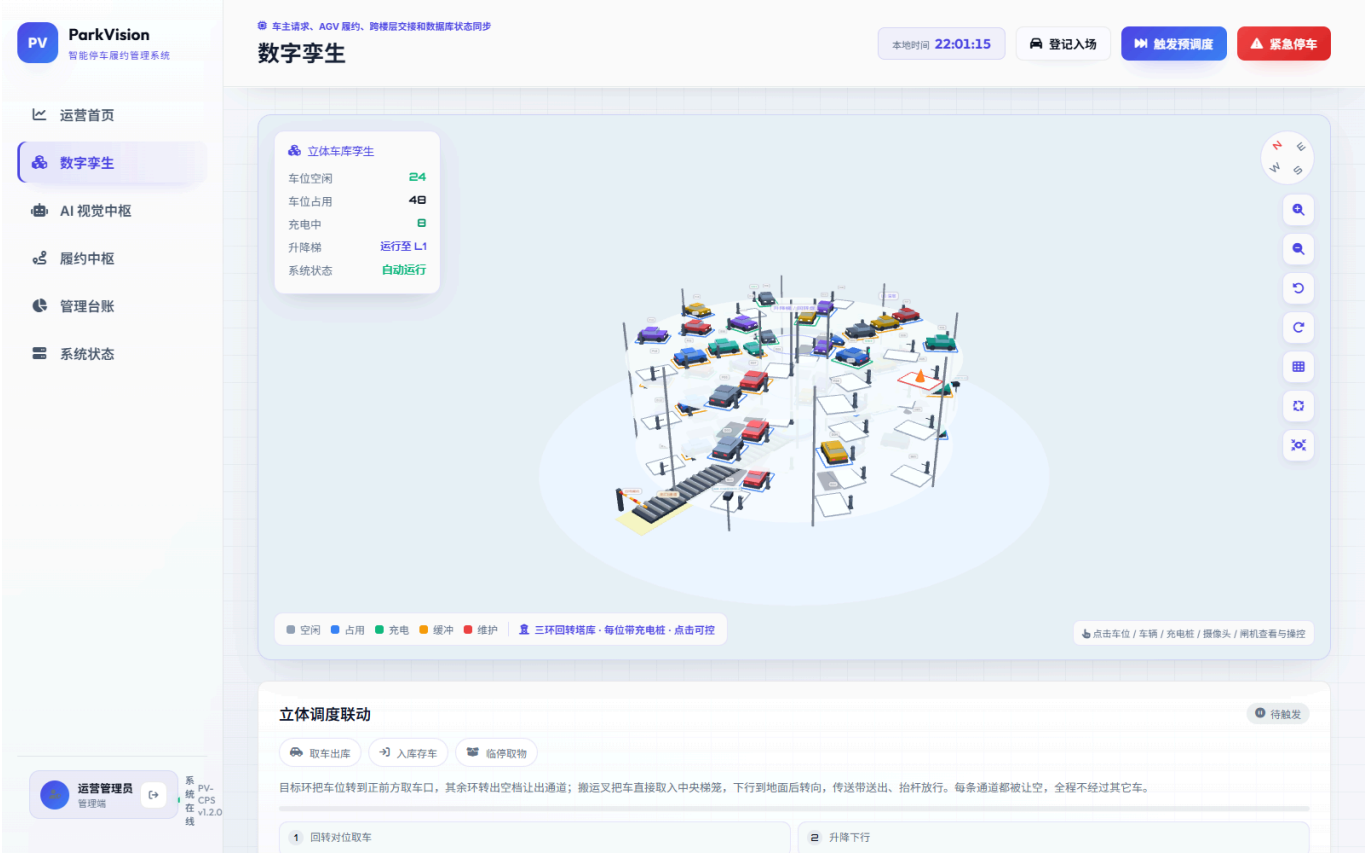


图 5-5 数字孪生实时页面

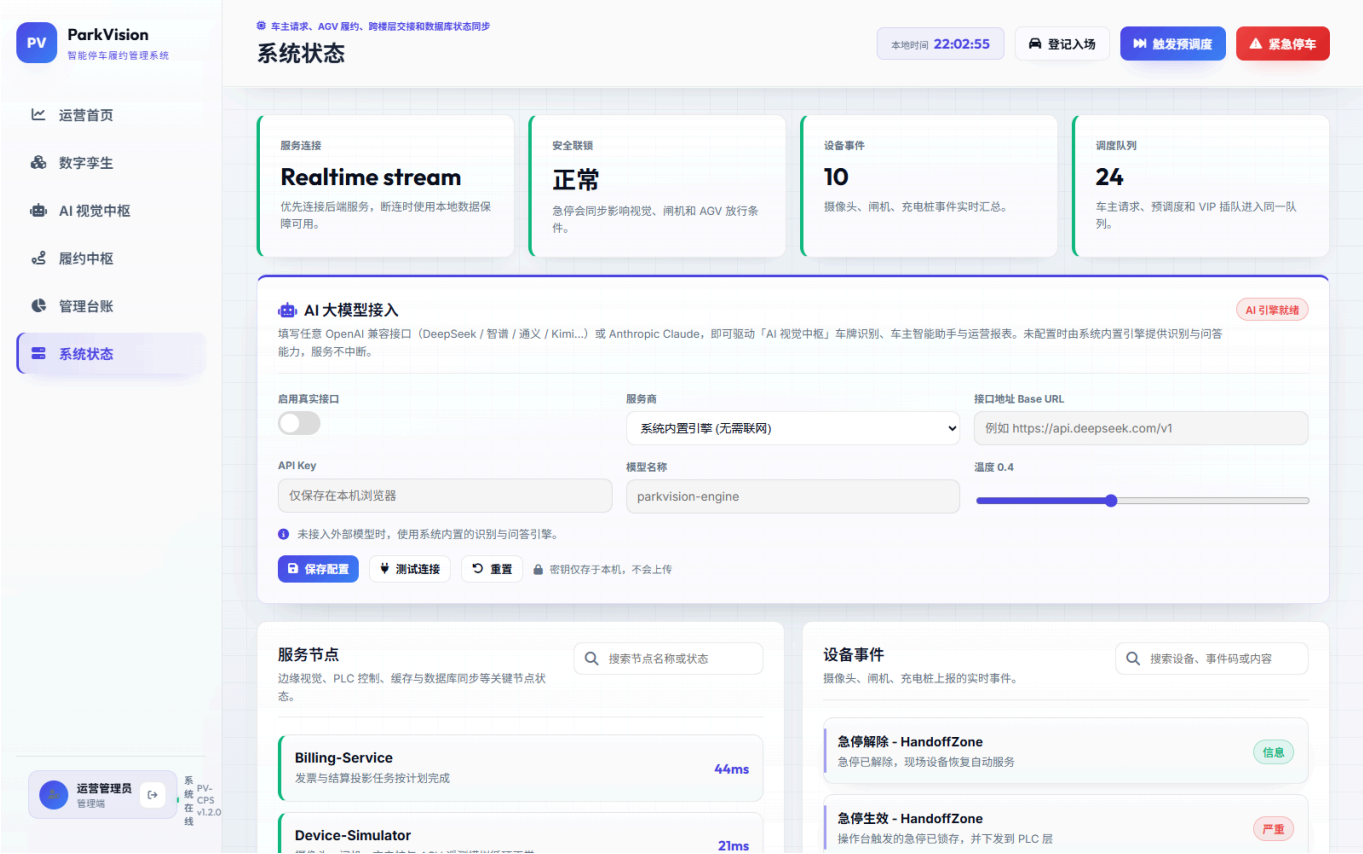


图 5-6 系统状态页面

这个组件的意义在于，它打通了“看得见”和“能管理”之间的鸿沟。过去很多系统只能在后台表格中看到数据，无法直观看到状态演化；而纯展示型大屏又缺乏与真实业务的同步。ParkVision 通过统一快照和实时流推送，使监控页面真正成为系统运行状态的可视化外壳。

5.6 车主端服务组件

虽然车主端功能在架构上分散于多个服务中，但从用户视角看，它已经形成了相对完整的移动服务链路。通过 OwnerController 与 OwnerService，车主可以完成：

- 1. 查看个人资料；
- 2. 查看名下车辆；
- 3. 查看历史与当前订单；
- 4. 发起存车、取车与临时取物；
- 5. 查询钱包与账单；
- 6. 充值钱包；
- 7. 管理预约；
- 8. 使用 AI 助手与导航能力。

这说明系统并不只是偏后台的运维管理系统，而是真正具备一定 C 端服务能力的应用系统。



图 5-7 车主预约页面



图 5-8 车主智能助手页面

车主端的完成度决定了系统是否真正具备面向用户交付的可能性。如果一个停车系统只能让管理员在后台操作，而无法让车主完成自助查询、预约、支付与咨询，那么它更接近管理工具，而不是完整产品。当前版本已经让车主侧形成相对闭环，这是项目成果中很有分量的一部分。

6. 关键业务流程说明

6.1 车辆入场流程

车辆入场是整个业务闭环的起点，当前系统入场流程如下：

1. 用户或管理员发起入场请求。
2. 系统校验车牌是否为空。
3. 系统检查该车牌是否已有未完成订单。
4. 系统选择一个可用车位。
5. 系统根据车辆属性将车位状态切换为 **OCCUPIED** 或 **CHARGING**。
6. 系统创建 **ParkingOrder** 并写入数据库。
7. 系统创建入场调度任务并记录设备事件。
8. 前端页面更新订单状态、车位状态与调度队列。

这一流程同时贯穿了用户输入校验、状态机初始化、仓储持久化与调度联动，体现出系统主链路的完整性。

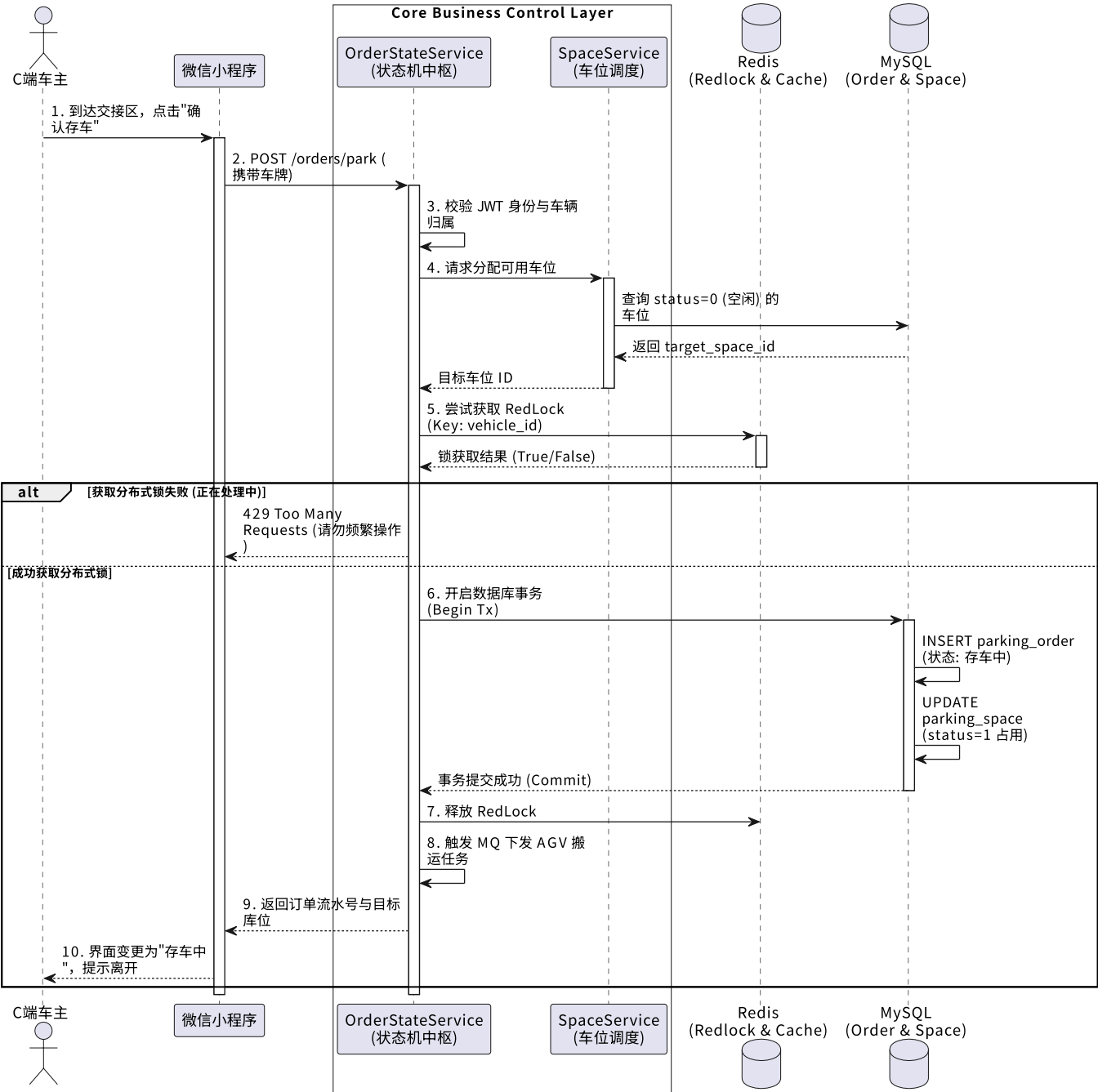


图 6-1 发起存车请求时序图

6.2 标准取车流程

标准取车流程如下：

- 1. 车主或管理员发起取车请求。
- 2. 系统根据订单号查找当前订单。
- 3. 系统将订单状态切换为 RETRIEVING。
- 4. 系统将对对应车位状态切换为 BUFFER。
- 5. 系统创建一条标准取车调度任务。
- 6. 调度服务将任务放入队列。
- 7. 调度 worker 自动分配空闲 AGV，并周期性推进任务。
- 8. 任务完成后，车辆到达交接区，等待支付结算。

在标准取车场景中，系统最需要解决的问题不是“能不能创建一条任务”，而是“任务创建后状态是否持续一致”。因此，取车流程的设计中必须同时考虑订单状态、车位状态、调度任务状态和 AGV 运行状态的同步变化。当前实现通过服务层集中处理，把这种多实体联动控制在同一条主链上。

6.3 临时取物流程

临时取物是系统支持柔性业务的重要体现，其流程为：

1. 车主对进行中的订单发起 **Touch-and-Go** 请求。
2. 系统将订单状态切换为 **TOUCHING**。
3. 系统将车位状态切换为 **BUFFER**。
4. 系统生成临时取物调度任务。
5. 任务完成后，车辆短时可达交接区。
6. 订单并不立即关闭，后续仍可返回停车状态或继续其他流程。

这个功能说明系统并非只支持简单的“入场—出场”二元过程，而是具备更接近现实停车业务的中间状态处理能力。

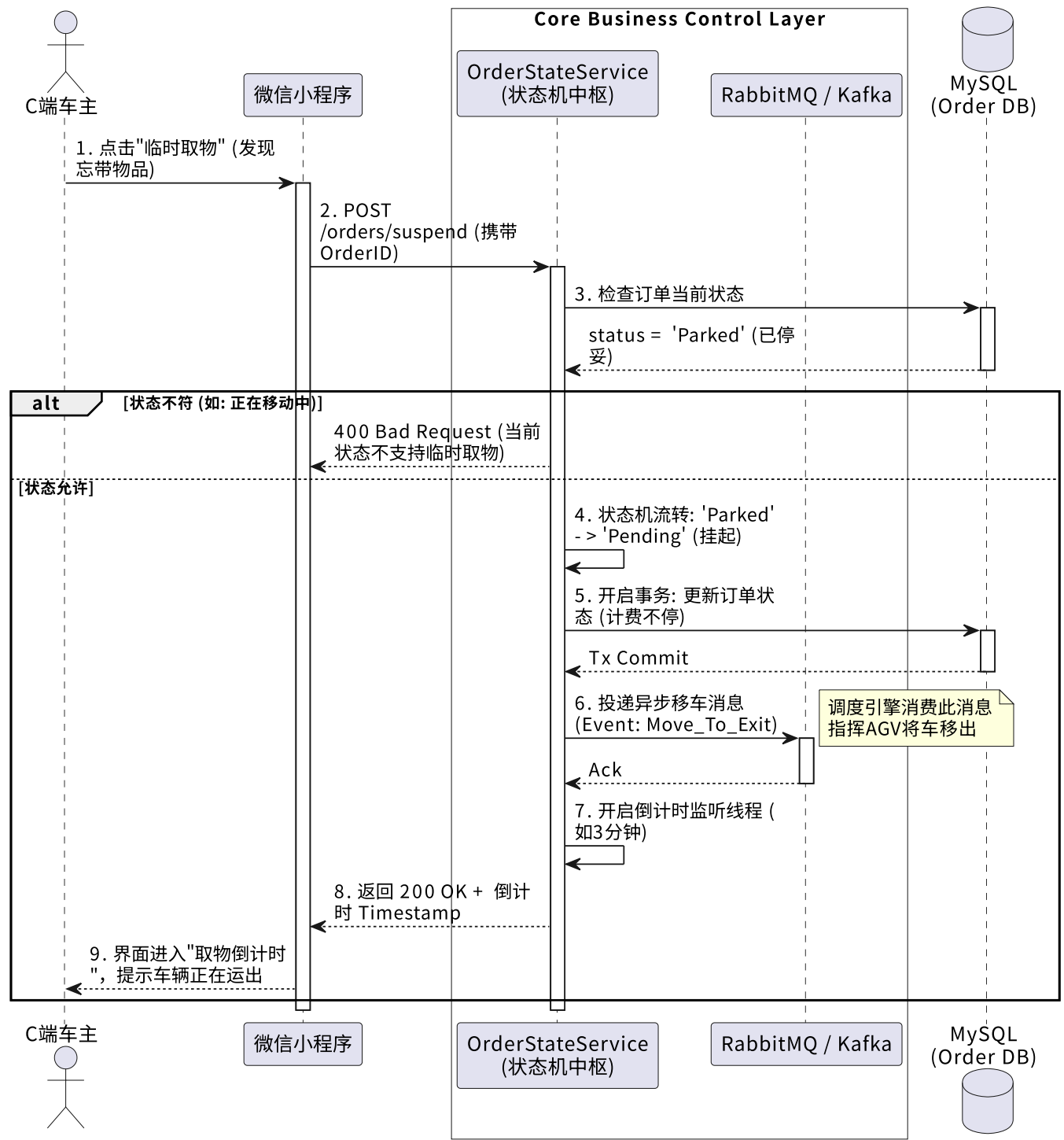


图 6-2 临时取物时序图

6.4 支付结算流程

支付结算流程如下：

- 1. 用户发起支付请求。
- 2. 系统将订单状态切换为 **FINISHED**。
- 3. **BillingService** 根据规则生成结算结果。
- 4. 系统保存支付记录与账单明细。
- 5. 车位状态被切换为 **EMPTY**。
- 6. 系统记录订单关闭事件。
- 7. 后台、车主端和数字孪生视图同步更新。

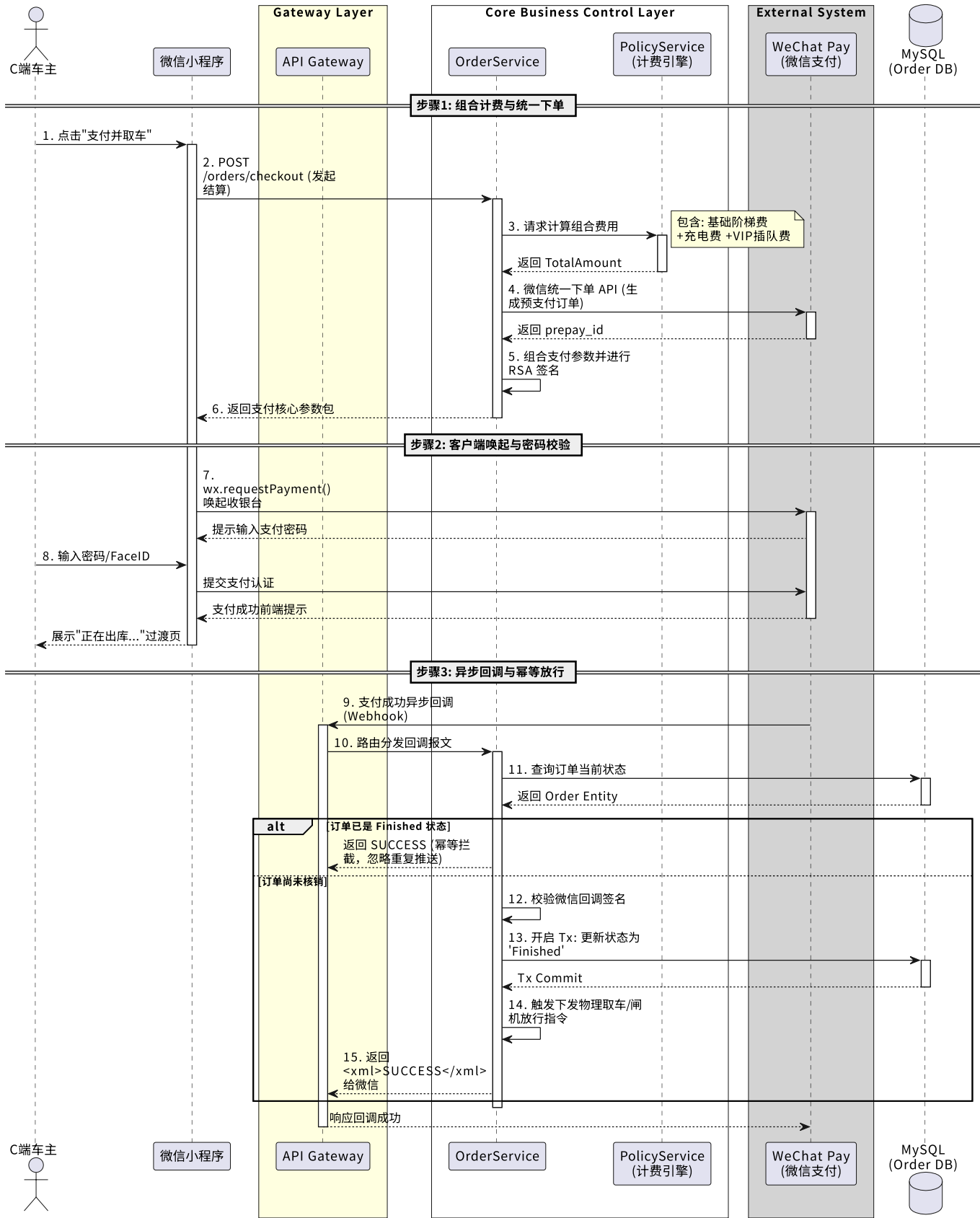


图 6-3 支付与复杂计费时序图

6.5 预约锁位流程

预约锁位流程如下：

1. 车主发起预约请求。
2. 系统记录预约信息并锁定目标车位。

3. 车主可以在预约有效期内取消预约。
4. 若车主按时到场，则 `ReservationService.fulfill()` 调用 `OrderService.admitToSlot()` 完成履约。
5. 若超过有效时间未履约，则定时任务自动将预约标记过期并释放车位。

这保证了预约能力不是一个“只显示预约记录”的前端功能，而是与真实车位资源绑定的后端业务能力。

预约能力的落地还意味着系统必须在“用户体验”和“资源利用率”之间取得平衡。若预约只是一个前端提示，不影响真实资源状态，那么它无法为用户带来确定性；但如果预约长期占用资源而缺乏过期释放机制，又会损害整体吞吐效率。当前版本通过预约过期任务和履约逻辑，在两者之间建立了相对合理的平衡。

6.6 AI 准入判定流程

AI 准入流程如下：

1. 前端上传图像或触发闸口识别请求。
2. `PlateRecognitionService` 输出识别结果。
3. `AccessControlService` 查询车牌在名单中的归属。
4. 系统根据判定结果得出 `ALLOW`、`REVIEW` 或 `DENY`。
5. 若允许且为闸口入场请求，则自动创建入场订单。
6. 若为拒绝场景，则生成告警记录。
7. 若为入侵场景，则返回 `ESTOP_AND_REVIEW` 并联动急停。

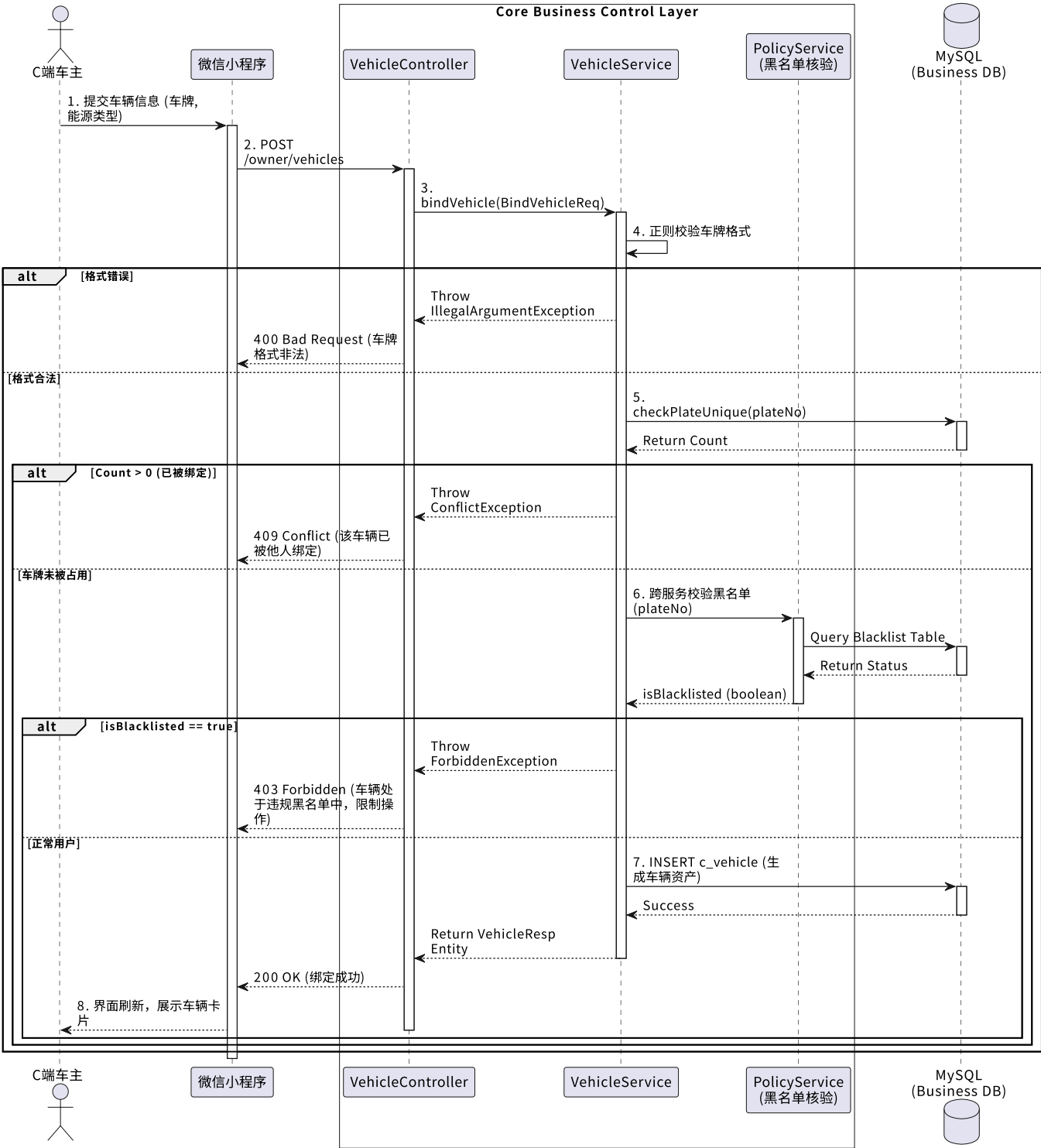


图 6-4 AI 准入与黑名单判定时序图

6.7 数字孪生实时同步流程

数字孪生同步流程如下：

1. 调度服务、设备服务、订单服务持续更新底层状态。
2. `TwinBroadcastService` 周期性构建统一快照。
3. `TwinStreamHub` 将快照广播给已连接页面。
4. 前端 `TwinView`、系统状态页和相关卡片组件接收最新状态。
5. 页面更新车位颜色、AGV 坐标、队列状态、告警提示和急停标识。

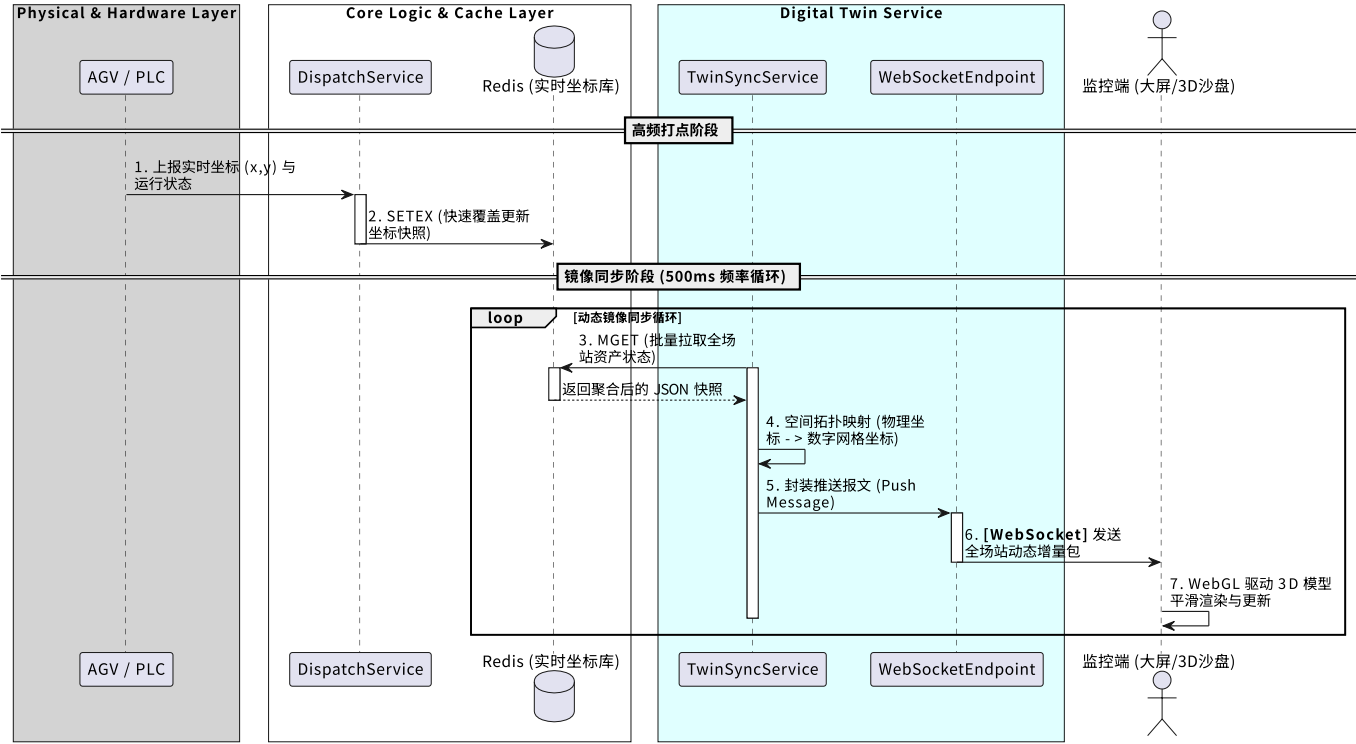


图 6-5 数字孪生坐标同步时序图

对于数字孪生来说，最大的难点不在于界面渲染，而在于后端是否能够持续提供可信的状态源。当前实现通过统一快照构建，把原本分散在不同业务模块中的状态重新组织后再对外广播，使前端可以在相对简洁的协议下消费到完整场站状态，这也是系统监控能力形成闭环的关键一步。

7. 关键技术实现与工程化方案

7.1 JWT 认证与角色隔离

系统采用 JWT 进行无状态认证。`JwtService` 会在登录成功后生成包含用户名、角色与显示名的令牌；`JwtAuthFilter` 则在请求进入业务处理前解析令牌、恢复认证上下文。配合 `SecurityConfig` 中的路径权限限制，可以将车主与管理员访问范围严格区分开来。

当前实现中，车主不能访问管理接口，管理员也不会进入车主端受限视图。相比仅在前端隐藏菜单的做法，这种服务端权限控制更符合真实系统的安全要求。

操作审计日志

记录每一次管理端写操作：操作账号、动作、目标接口与结果状态。

搜索账号、方法、路径或状态

时间	账号	动作	目标	结果
2026-06-10 22:46:13	- ROLE_ANONYMOUS	POST	/api/auth/login	200
2026-06-10 22:45:46	- ROLE_ANONYMOUS	POST	/api/auth/login	200
2026-06-10 22:44:50	- ROLE_ANONYMOUS	POST	/api/auth/login	200
2026-06-10 22:44:13	- ROLE_ANONYMOUS	POST	/api/auth/login	200
2026-06-10 22:43:48	- ROLE_ANONYMOUS	POST	/api/auth/login	200
2026-06-10 22:02:33	admin ROLE_ADMIN	POST	/api/admin/report	200
2026-06-10 22:02:30	- ROLE_ANONYMOUS	POST	/api/auth/login	200
2026-06-10 22:02:30	- ROLE_ANONYMOUS	POST	/api/auth/login	200
2026-06-10 22:01:18	admin ROLE_ADMIN	POST	/api/devices/emergency	200

图 7-1 后台审计页面

7.2 服务层统一状态入口

系统并没有把状态变更分散到多个控制器中，而是集中在服务层处理。例如：

- 1. 入场逻辑由 `OrderService` 统一负责；
- 2. 预约履约通过 `ReservationService` 调用订单服务；
- 3. 调度任务推进由 `DispatchService` 统一负责；
- 4. 费用计算通过 `PricingService` 与 `BillingService` 保持一致；
- 5. 视觉识别结果在 `VisionService` 中统一转化为业务动作。

这种设计可以显著减少跨页面、跨接口造成的数据失真问题，也是后端可维护性的基础。

服务层统一状态入口的另一个优势，在于便于后续接入更多业务分支。无论未来新增会员权益、更多告警类型还是更细粒度的订单阶段，只要仍然围绕统一服务层进行状态扭转，就不会破坏系统现有的主链路一致性。

7.3 审计日志机制

`AuditInterceptor` 会自动捕捉所有 `POST`、`PUT`、`PATCH`、`DELETE` 请求，并记录：

- 操作者用户名；
- 操作者角色；
- 请求方法；
- 请求路径；
- HTTP 状态码；
- 客户端 IP；
- 发生时间。

对于后台管理系统而言，这一机制非常重要。它不仅有助于复盘异常操作，也有利于后续做合规审计、操作追责和风险分析。

后端优化代码素材 1

用于展示 JWT 鉴权、角色权限控制与后台写操作审计能力。

JWT 鉴权与角色权限

backend/src/main/java/com/parkvision/cps/config/SecurityConfig.java

后端优化

```
28  @Bean
29  public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
30      http
31          .csrf(AbstractHttpConfigurer::disable)
32          .cors(Customizer.withDefaults())
33          .sessionManagement(session → session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
34          .headers(headers → headers.frameOptions(frame → frame.disable()))
35          .authorizeHttpRequests(auth → auth
36              .requestMatchers("/api/auth/**").permitAll()
37              .requestMatchers("/api/twin/**").permitAll()
38              .requestMatchers("/actuator/**", "/h2-console/**", "/error").permitAll()
39              .requestMatchers("/api/admin/**", "/api/system/**").hasRole("ADMIN")
40              .requestMatchers("/api/**").authenticated()
41              .anyRequest().permitAll()
42          )
43          .exceptionHandling(handling → handling
44              .authenticationEntryPoint((request, response, ex) →
45                  writeJsonStatus(response, HttpServletResponse.SC_UNAUTHORIZED, "AUTH_REQUIRED", "需要登录后访问"))
46              .accessDeniedHandler((request, response, ex) →
47                  writeJsonStatus(response, HttpServletResponse.SC_FORBIDDEN, "FORBIDDEN", "当前角色无权访问该资源"))
48          )
49          .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
50      return http.build();
```

写操作审计日志落库

backend/src/main/java/com/parkvision/cps/config/AuditInterceptor.java

后端优化

```
30  @Override
31  public void afterCompletion(HttpServletRequest request, HttpServletResponse response, Object handler, Exception ex) {
32      if (!MUTATING.contains(request.getMethod())) {
33          return;
34      }
35      String path = request.getRequestURI();
36      // Skip the realtime/stream channels and auth probes that aren't business actions.
37      if (path == null || path.startsWith("/api/twin/stream")) {
38          return;
39      }
40
41      Authentication auth = SecurityContextHolder.getContext().getAuthentication();
42      String username = auth != null && auth.isAuthenticated() && !"anonymousUser".equals(auth.getName())
43          ? auth.getName() : "-";
44      String role = auth != null && auth.getAuthorities() != null && !auth.getAuthorities().isEmpty()
45          ? auth.getAuthorities().iterator().next().getAuthority() : "-";
46
47      try {
48          repository.saveAuditLog(new AuditLog(
49              null, username, role, request.getMethod(), path,
50              response.getStatus(), clientIp(request), LocalDateTime.now()
51          ));
52      } catch (Exception ignored) {
53          // Auditing must never break the request lifecycle.
54      }
```

图 7-2 审计与安全实现代码截图

7.4 规则驱动的费用引擎

系统计费并不是写死的固定金额，而是通过 `PricingRule` 规则实体驱动。当前已支持：

1. 免费时长；
2. 首小时费用；
3. 续停小时费用；
4. 高峰倍率；
5. 单次封顶；
6. 充电附加费；
7. VIP 优先附加费。

同时，费用预览与最终结算共用一套计算口径，这对于避免用户端金额与后台账单不一致具有重要意义。

这套计费逻辑的现实价值在于，它已经超出了简单“停车时长乘单价”的静态规则，而开始具备策略化与解释能力。用户可以看到费用构成，后台可以调整规则，系统可以根据不同订单状态叠加不同附加项，从而使计费既可配置，又能保持统一口径。

7.5 实时推送与定时任务结合

系统利用 `@Scheduled` 实现了多项周期任务：

1. 调度队列推进；
2. 预约过期检查；
3. 设备状态更新；
4. 数字孪生快照广播；
5. 告警监控。

这说明系统的状态演化不是完全依赖用户点击触发，而是包含“系统自身持续推进”的逻辑，这正是智能系统与静态 CRUD 系统的重要区别。

实时流 / 轮询结合机制代码素材

用于展示 App 启动时接入 SSE 实时流，并以 5 秒轮询作为兜底同步机制。

应用启动时同时接入实时流与轮询任务

frontend/src/App.vue

实时流 + 轮询

```
22 onMounted(async () => {
23   await hydrate();
24   connectTwinStream();
25   timer.value = window.setInterval(() => {
26     void pollRealtime();
27   }, 5000);
28 });
29
30 onUnmounted(() => {
31   if (timer.value) window.clearInterval(timer.value);
32   disconnectTwinStream();
```

SSE 实时流接收 + 轮询兜底状态切换

frontend/src/stores/parkingStore.js

实时流 + 轮询

```
294 export async function pollRealtime() {
295   try {
296     await refreshCore();
297   } catch {
298     state.onlineMode = "Fallback mode";
299   }
300   ...
301   ...
313 /* ----- *
314 * Realtime digital-twin stream (Server-Sent Events) *
315 * The backend pushes authoritative state (slots / summary / queue / *
316 * AGV / emergency) sub-second, so the twin and dashboard reflect the *
317 * real database without waiting for the slower poll. EventSource has *
318 * built-in auto-reconnect; polling stays as a fallback while the *
319 * stream is unavailable. *
320 * ----- */
321 let twinStream = null;
322
323 function resolveTwinStreamUrl() {
324   const base = import.meta.env.VITE_API_BASE_URL || "/api";
325   return `${base.replace(/\/$/, "")}/twin/stream`;
326 }
327
328 function applyTwinSnapshot(snapshot) {
329   if (!snapshot) return;
330   if (snapshot.summary) state.summary = snapshot.summary;
331   if (Array.isArray(snapshot.slots)) state.slots = snapshot.slots;
332   if (Array.isArray(snapshot.agvs)) state.agvs = normalizeAgvs(snapshot.agvs);
333   if (Array.isArray(snapshot.queue)) state.queue = snapshot.queue;
334   reapplyReservations();
335   if (typeof snapshot.emergency === "boolean") state.emergency = snapshot.emergency;
336   state.activePlate = getters.currentOrder.value?.plateNo || state.activePlate;
337 }
338
339 function onTwinMessage(event) {
340   try {
341     applyTwinSnapshot(JSON.parse(event.data));
342     state.onlineMode = "Realtime stream";
343   } catch {
344     /* ignore malformed frame */
345   }
346 }
347
348 export function connectTwinStream() {
349   if (typeof window === "undefined" || !("EventSource" in window)) return;
```

```
350 if (!twinStream) return;
351 try {
352     twinStream = new EventSource(resolveTwinStreamUrl());
353 } catch {
354     twinStream = null;
355     return;
356 }
357 twinStream.addEventListener("twin", onTwinMessage);
358 twinStream.onmessage = onTwinMessage;
359 twinStream.onerror = () => {
360     // EventSource reconnects automatically; show that we fell back to polling.
361     if (state.onlineMode === "Realtime stream") state.onlineMode = "Backend connected";
362 };
363 }
```

图 7-3 实时流推送相关代码截图

7.6 AI 能力接入策略

AI 能力在当前系统中采用配置化接入方式：

1. 对话模型通过 `PARKVISION_CHAT_PROVIDER` 等环境变量进行切换。
2. OCR/车牌识别可在内置引擎、HyperLPR、本地百度 OCR 等模式间切换。
3. AI 对话调用由服务端代理完成，避免 API 密钥暴露给前端。

这一做法有两个现实意义：第一，项目可以在不同部署环境中灵活切换模型来源；第二，前端页面不需要感知底层模型细节，只需要消费统一接口。

从工程上看，这种配置化接入方式降低了 AI 能力与核心业务之间的耦合。系统未来可以替换更强的 OCR 服务、更稳定的大模型接口，甚至切换为本地推理方案，而不会对前端页面和大部分业务代码造成结构性影响。

7.7 持久化与未来演进兼容

系统当前默认使用 H2 文件数据库，这是为了在单机环境中获得足够低的部署成本与较高的可复现性。但系统并没有把持久化逻辑写死在 H2 上，而是：

1. 提供统一仓储接口；
2. 提供 H2 与 MySQL 双 schema；
3. 在配置文件中暴露数据源与平台切换参数；
4. 通过 JDBC 模式保证业务数据真正落盘。

因此，系统未来迁移到 MySQL 并不需要推翻业务层逻辑，只需要替换底层配置和适配部分 DDL/数据初始化逻辑即可。

这也是项目能够顺利完成服务器部署的重要原因之一。因为只要底层配置和打包方式明确，系统就可以从“开发机本地运行”自然过渡到“服务器持续运行”，而不需要对业务逻辑进行大规模重写。

8. 测试与验证结果

8.1 测试目标

测试工作的目标不是简单验证页面能否打开，而是验证：

- 1. 关键业务逻辑是否能够正确执行；
- 2. 典型异常场景是否能够被拦截；
- 3. 状态流转是否符合预期；
- 4. 前后端状态组织是否具备基本稳定性。

测试阶段不仅是为了证明“功能能跑通”，更重要的是验证各模块是否在连续操作和多次重复操作后仍然保持一致性。对于停车系统这样的状态型应用来说，真正有价值的测试结论来自对主链路和异常链路的共同覆盖，而不是单次成功截图本身。

8.2 后端自动化测试

项目当前后端测试位于 `backend/src/test/java/com/parkvision/cps/` 下，已覆盖应用启动、订单服务、调度服务、后台服务和告警监控等逻辑。

在 2026-06-14 实际执行：

```
cd backend
.\mvnw.cmd test
```

测试结果如下：

测试类	测试数量	结果
ParkVisionApplicationTests	1	通过
AdminServiceTest	8	通过
AlertMonitorServiceTest	3	通过
DispatchServiceTest	3	通过
ExperienceServiceTest	2	通过
OrderServiceTest	2	通过
合计	19	全部通过

后端自动化测试覆盖的重点集中在服务层，因为服务层正是业务规则和状态变更的核心发生地。测试通过说明当前订单、调度、告警与后台聚合逻辑已经具备较好的基础稳定性，也为系统后续继续扩展提供了较安全的迭代起点。

8.3 前端自动化测试

前端当前使用 Vitest 对状态仓储进行基础测试。在 2026-06-14 实际执行：

```
cd frontend
npm test
```

测试结果如下：

测试文件	测试数量	结果
src/stores/parkingStore.test.js	2	通过
合计	2	全部通过

前端测试数量目前少于后端，这是当前项目仍需继续加强的一点。不过，现有测试已经至少验证了状态仓储层的关键行为，这对于多页面共享状态的前端系统来说仍具有一定意义。后续若继续完善，可优先补充登录、角色跳转、预约提交流程和孪生流接收逻辑的测试覆盖。

8.4 联调与功能验证场景

除自动化测试外，系统在实际联调中重点验证了以下场景：

验证场景	验证内容	验证结论
登录与注册	管理员登录、车主注册、角色跳转	正常
车辆绑定与个人数据	车主查看个人资料、车辆与订单	正常
入场建单	指定车牌入场、重复入场拦截、车位联动	正常
取车流程	标准取车、调度任务生成、车位状态切换	正常
临时取物	订单挂起、缓冲车位切换	正常
预约锁位	创建、取消、履约、过期释放	正常
动态计费	预览费用、账单拆分、支付结算	正常
AI 准入	识别、名单判定、拒绝/复核/放行	正常
数字孪生	快照推送、AGV 与车位实时显示	正常
急停控制	入侵场景联动与人工恢复	正常

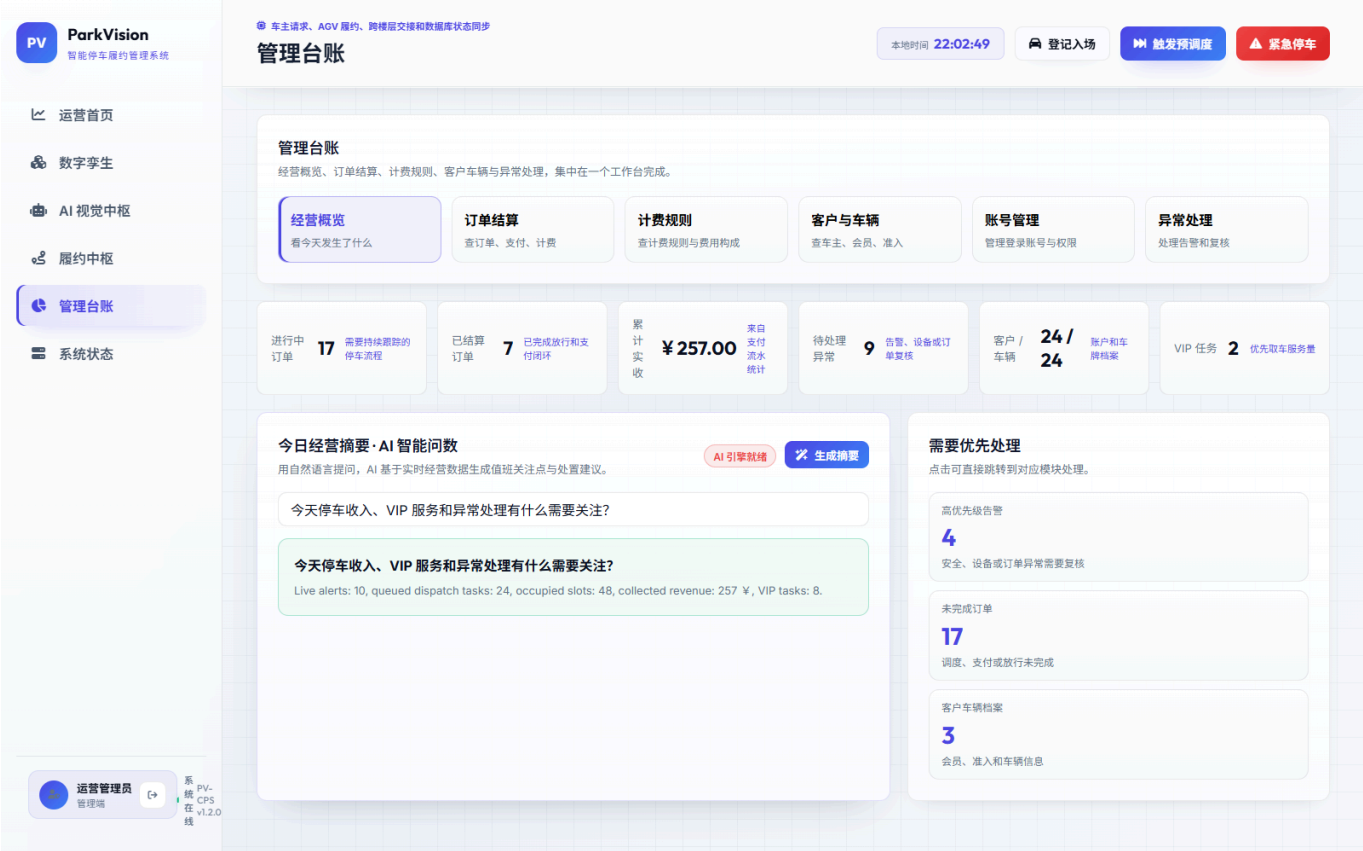


图 8-1 后台综合运营页面

8.5 测试结论

从当前测试与联调结果看，系统已经具备较好的功能完整性和模块协作能力。虽然自动化测试的广度仍可继续提升，但关键业务主链路已经经过实际运行验证，系统在单机场景下具备较强的稳定性与可复现性。

再结合服务器部署情况来看，系统验证已经不局限于开发环境本身，而是进一步覆盖到了打包、部署和远程访问这一层。这意味着结题时的项目成果不仅可在本地复现，也已经具备在独立运行环境中稳定承载前后端服务的能力。

9. 项目成果、亮点与工程价值

9.1 完整的业务闭环能力

ParkVision 的首要成果并不在于页面数量，而在于它已经形成了完整的业务闭环：从用户注册登录、车辆绑定、入场建单、调度执行、取车结算，到后台审计、数字孪生展示和 AI 准入判定，系统核心链路是可串联、可追踪、可验证的。

这种完整性使系统不再只是若干“孤立功能点”的集合。即使将任何一个模块单独拿出来看，例如 AI 识别、预约锁位或系统监控，它们也都能被放回整体业务链路中解释其输入、输出与作用位置。这是项目从功能堆叠走向系统化实现的重要标志。

9.2 多角色统一平台能力

系统同时面向车主、管理员、调度人员和安全值守人员，不同角色在统一系统中完成不同职责，体现了面向真实场站管理的组织化能力。这一点对于后续从课程项目升级为可用原型十分关键。

多角色统一平台的现实价值还体现在：所有角色都围绕统一的数据源和统一的状态模型进行工作，而不是各自维护一套脱节的信息视图。这样既降低了协作成本，也为后续实现更细粒度的权限与流程控制提供了基础。

9.3 状态驱动而非页面驱动

项目最有价值的工程实践之一，是围绕服务层状态流转而不是围绕页面按钮编写逻辑。订单、车位、调度、设备、计费 and AI 决策在服务端统一建模，这是系统可扩展性的基础。

这也意味着项目具备较好的可维护性。当业务需求发生变化时，开发者首先需要调整的是状态规则和服务逻辑，而不是在多个页面中分别修补 UI 行为。对于后续长期维护而言，这种方式比页面驱动式开发更加稳健。

9.4 数字孪生与业务状态绑定

许多系统的大屏只是图形化展示，而 ParkVision 的数字孪生由后端统一快照驱动，能与车位、AGV、队列和急停状态同步变化，因而具有更高的真实性和解释力。

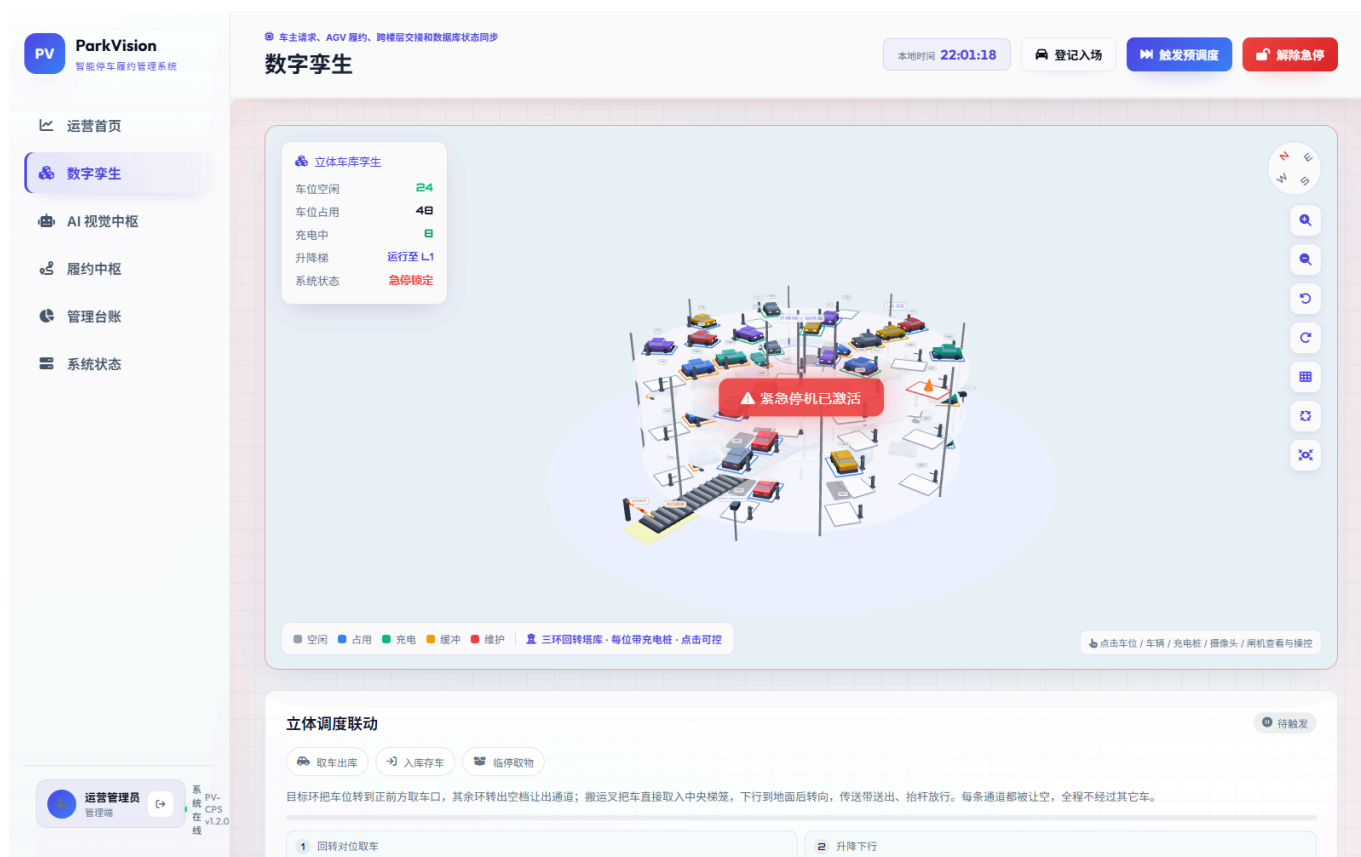


图 9-1 数字孪生应急联动页面

9.5 AI 能力与业务规则融合

AI 能力没有以“黑盒替代规则”的方式接入，而是与名单判定、人工复核和急停联动结合起来，体现出更符合真实业务治理需求的设计取向。

9.6 工程结构清晰，便于后续演进

仓储接口、H2/MySQL 双 schema、配置化模型接入、SSE 推送、统一启动脚本和 Docker Compose 草案，都表明系统不仅考虑到了“做出来”，也考虑到了“如何持续维护与演进”。

特别是在服务器部署完成后，这一点体现得更加明显。一个真正考虑演进性的系统，不应在离开开发者电脑后就失去运行能力；而 ParkVision 当前已经完成从本地运行到服务器运行的跨越，这使其工程价值更具说服力。

10. 已知不足与后续改进方向

虽然 ParkVision 已经具备较完整的单场站软件能力，但从更高标准的工程化与生产可用性角度看，系统仍有若干可以继续改进的地方。

10.1 数据库与中间件层仍偏单机化

当前系统默认使用 H2 文件数据库，优点是轻量、方便、可快速启动，但其在并发能力、集群容灾和运维生态上与 MySQL 等生产级数据库仍有差距。未来若系统进入真实部署阶段，应优先完成：

1. MySQL 迁移；
2. Redis 缓存与分布式锁引入；
3. 消息队列解耦长耗时调度任务；
4. 多实例部署下的状态一致性治理。

这里的不足并不意味着当前方案不可用，而是说明系统在规模扩大后会遇到新的边界。当前 H2 模式适合单机与中小规模验证，但当部署环境进入高并发、持续在线和多用户同时操作场景后，更标准的数据库与缓存体系会成为必要条件。

10.2 后端当前为模块化单体，不是分布式微服务

当前系统的模块边界清晰，但仍运行在同一个 Spring Boot 进程内。对于课程项目而言，这种方案合理且稳定；但若场站规模扩大，或需要引入更多 AI 计算与外部设备网关，未来可以考虑按组件将系统拆分为多个独立服务。

从另一角度看，当前采用模块化单体也是一种务实选择。它保证了项目在现阶段先把业务主链路跑通，并将复杂度控制在可维护范围内。后续如果确有必要拆分，也可以基于现有组件边界逐步推进，而不必推翻既有实现。

10.3 真实设备联调仍需进一步深化

系统已经完成 AGV、摄像头、闸机、充电桩等设备实体与协议风格接口的抽象，但与真实硬件设备的现场联调仍需要继续推进。后续应重点完成：

1. 与真实 PLC / AGV 网关的双向通讯接入；
2. 设备异常码与告警码映射标准化；
3. 弱网场景下的数据重传与断线恢复机制；
4. 设备级权限与安全闭锁策略增强。

10.4 安全能力仍可增强

目前系统已具备 JWT、角色隔离和审计日志能力，但若面向更正式的生产环境，还应补足：

1. 刷新令牌与令牌失效机制；
2. 更精细的 RBAC 权限粒度；
3. 接口限流与登录风控；
4. 更严格的 CORS 与密钥管理；

5. 更系统化的日志、指标和链路追踪方案。

此外，服务器部署后对安全要求会进一步提高。因为系统一旦脱离本地开发环境并面向更多使用者开放，登录安全、密钥管理、访问边界和异常行为识别的要求都会显著提升。因此，本章提出的增强方向既是技术优化项，也是后续走向更大范围使用时必须完成的基础工程。

10.5 AI 可信治理与模型效果提升空间

当前 AI 能力已经能够支撑基础准入判定和问答，但若用于更复杂或更高风险环境，还需继续提升：

- 1. 视觉识别在复杂天气和特殊光照下的鲁棒性；
- 2. 车流预测模型的时序准确率；
- 3. LLM 输出的稳定性与可控性；
- 4. AI 决策过程的规则解释和人工复核工具链。

11. 部署与运行说明

11.1 开发运行环境

根据当前项目配置，系统建议运行在以下环境中：

环境项	建议版本
JDK	17
Node.js	可运行 Vite 6 的版本
Maven	可选，仓库已内置 Maven Wrapper
Python	用于本地 OCR 服务，可选
操作系统	Windows 开发环境已验证

虽然表中给出的环境主要针对本地开发，但项目在结题阶段已经完成服务器部署，因此运行环境实际上已经覆盖了“开发环境 + 部署环境”两类场景。开发环境强调快速复现和调试效率，部署环境则更强调打包产物稳定性、配置隔离和持续运行能力。

11.2 本地启动方式

推荐在项目根目录执行：

```
.\start.ps1
```

该脚本会自动完成以下工作：

- 1. 检查 Node.js 与 npm 环境；
- 2. 安装前端依赖（如需要）；
- 3. 启动本地 OCR 服务；
- 4. 启动 Spring Boot 后端；
- 5. 启动 Vue 前端开发服务器；

6. 将日志输出到 `logs/` 目录。

本地一键启动方式更适合作为开发、调试和课程演示的入口。它的优势在于依赖集中、流程清晰、成员之间复现成本低。对于日常开发来说，这种方式可以保证团队在相同端口、相同目录结构和相近数据基线上展开协作。

11.3 单独启动方式

若需要手动分离启动，可分别执行：

前端：

```
cd frontend
npm install
npm run dev
```

后端：

```
cd backend
.\mvnw.cmd spring-boot:run
```

后端测试：

```
cd backend
.\mvnw.cmd test
```

前端测试：

```
cd frontend
npm test
```

11.4 服务器部署情况

截至结题阶段，ParkVision 已经完成服务器部署。当前部署采用前后端分离方式：前端通过构建后的 `dist` 静态资源部署到服务器 Web 服务中，后端通过打包后的 `parkvision-backend-*.jar` 在服务器上运行，并通过配置项注入数据库连接、JWT 密钥、AI 服务和 OCR 接入参数。这样做使系统摆脱了对开发机本地环境的依赖，能够以更稳定的方式对外提供访问。

从仓库脚本可以看出，服务器部署流程已经被较好地规范化。`backend/scripts/package.ps1` 负责生成可部署后端 jar 与前端构建产物，`backend/scripts/start-prod.ps1` 负责以后端生产化配置启动服务，并支持通过环境变量切换数据库账号、数据源地址和 JWT 密钥。也就是说，项目不仅在概念上具备部署能力，而且已经形成了较完整的部署路径与运行基线。

服务器部署完成后的价值主要体现在三个方面。第一，系统能够持续运行，便于多人同时访问和远程演示；第二，前后端分离部署让页面发布与后端升级可以相对独立进行；第三，部署过程本身验证了系统的打包能力、配置能力和环境适配能力。对于结题文档而言，这说明项目已经从“开发完成”进一步走到了“交付运行”的阶段。

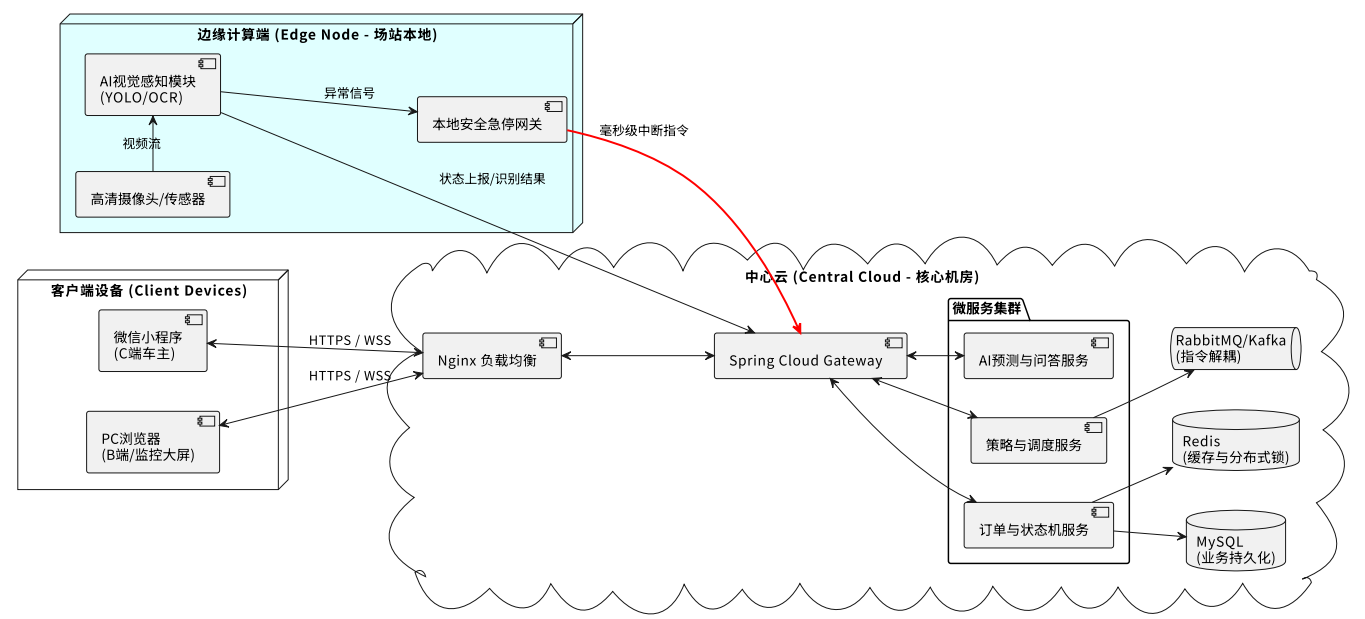


图 11-1 服务器部署与运行结构示意图

11.5 配置项说明

后端 `application.yml` 中的关键可配置项包括：

配置项	作用
<code>PARKVISION_DATASOURCE_URL</code>	数据源地址
<code>PARKVISION_SQL_PLATFORM</code>	初始化 schema 平台选择
<code>PARKVISION_JWT_SECRET</code>	JWT 密钥
<code>PARKVISION_CHAT_PROVIDER</code>	对话模型来源
<code>PARKVISION_LPR_PROVIDER</code>	OCR / 车牌识别来源
<code>PARKVISION_DEVICE_SIMULATION_ENABLED</code>	设备状态刷新开关
<code>PARKVISION_TWIN_BROADCAST_INTERVAL_MS</code>	孪生广播周期

12. 结题总结

ParkVision 智能停车系统项目已经完成了从概念方案到可运行软件系统的阶段性落地。系统不再停留于静态文档、页面草图或功能堆叠层面，而是围绕真实停车业务建立起了一套相对完整的软件控制闭环。当前版本已经具备多角色接入、订单状态机、调度策略、AI 准入、数字孪生展示、设备监控、动态计费与审计追踪等关键能力，并能够在单机场景下稳定运行。

更重要的是，项目在实现过程中没有把重点放在“页面展示丰富”上，而是通过统一状态模型、后端服务分层、规则驱动逻辑和实时推送机制，努力将系统构建为一个可以被持续维护和继续扩展的软件平台。这使得 ParkVision 不仅能够支撑课程项目结题，也具备了继续向更高工程化水平演进的基础。

结题阶段完成服务器部署后，项目的成果又向前推进了一步。系统不再局限于本地开发环境中的运行和演示，而是具备了在独立服务器环境中持续提供服务的能力。无论从项目复盘还是从后续扩展角度看，这都使 ParkVision 更接近一个真正可交付的软件系统。

从结题角度看，ParkVision 的主要成果可以概括为三点：

- 1. 建立了围绕立体车库业务的完整软件系统框架；
- 2. 实现了核心功能模块间的真实联动与数据闭环；
- 3. 形成了较清晰的后续扩展方向与工程化演进路径。

因此，本项目可以认为已经达成了课程综合项目在“问题抽象、系统设计、功能实现、工程验证、文档沉淀”几个维度上的预期目标。

附录 A：前端页面映射

页面	路径	面向角色	主要作用
登录页	/login	全体用户	统一登录与身份入口
运营首页	/	管理端	汇总 KPI、车流、事件与业务总览
数字孪生页	/twin	管理端 / 监控端	展示 AGV、车位、任务与场站状态
AI 视觉中枢	/ai	管理端 / 安全侧	展示识别、预测、准入与 AI 能力
履约中枢	/dispatch	调度侧	查看与推进调度队列
管理台账	/admin	管理端	订单、规则、用户、名单、报表与数据治理
系统状态页	/system	管理端 / 运维侧	查看节点、设备、急停与风险状态
车主端 H5	/owner	车主	车辆、订单、预约、钱包、导航、助手

附录 B：后端控制器清单

控制器	职责
AuthController	登录、注册、当前身份查询
OwnerController	车主资料、车辆、订单、预约、钱包、账单
OrderController	管理侧订单流转
DispatchController	调度队列、AGV、预调度、VIP
DashboardController	首页与统计数据
AiController	AI 对话与模型状态
VisionController	视觉识别与门禁联动
PricingController	计费预览与规则相关能力
ParkingController	车位相关查询与操作

控制器	职责
NavigationController	导航能力
AdminController	后台综合治理数据
AdminUserController	后台账户管理
DeviceController	设备总览、设备状态、急停控制
DeviceAdminController	设备管理相关接口
SystemController	系统节点与健康状态
TwinStreamController	数字孪生 SSE 流接口

附录 C：后端核心服务清单

服务类	主要职责
AuthService	登录、注册、JWT 发放
OwnerService	车主维度业务封装
OrderService	订单创建、状态变更、车位联动
ReservationService	预约创建、履约、取消、过期释放
DispatchService	队列调度、AGV 推进、预调度、VIP 取车
PricingService	计费规则解析与费用预览
BillingService	最终账单结算
VisionService	视觉识别、准入判定、门禁联动
AccessControlService	名单校验与放行策略
AiChatService	大模型代理调用
DashboardService	KPI 汇总、预测与报表数据
TwinBroadcastService	数字孪生快照构建与广播
DeviceService	设备遥测、设备事件、急停状态
AdminService	管理端聚合视图与业务支撑
AccountAdminService	后台账户治理
NavigationService	导航数据生成
AlertMonitorService	告警监测
ParkingService	车位域查询与支撑逻辑
PlateRecognitionService	OCR / 车牌识别封装